Machine Learning Principles: A Helicopter Tour

Christian Hansen - University of Chicago

Predictive Modeling Basics

Overview

Lecture Goal: Provide an overview of leading ML methods with a focus on supervised learning.

Before covering methods, we'll briefly **review** the core machinery of predictive modeling:

- Prediction loss and the bias/variance tradeoff
- In-sample vs. out-of-sample fit
- Validation and cross-validation

Predictive Models

Supervised Learning

A core statistical learning task is creating a rule for predicting the (as yet unseen) value of a target (Y) given observed characteristics X (i.e. prediction/forecasting).

Useful to think about relation between

- target: Y and
- p-dimensional predictor variable: $X = (X_1, ..., X_p)$

$$\underbrace{Y}_{\text{target}} = \underbrace{f(X)}_{\text{signal}} + \underbrace{\varepsilon}_{\text{noise}}$$

Goal: Learn $f(\cdot)$ from the data in a way that yields "generalizable" forecasts

- Produce a forecast rule that minimizes expected forecast loss
- Will focus on minimizing squared error loss: MSE = $\mathrm{E}[(Y - f(X))^2]$
  - Recall $\mathrm{E}[Y|X]$ provides the squared error loss minimizing prediction rule for Y
  - Nothing fundamentally different in classification problems (problems with categorical Y)

Setting

Assume target prediction rule is $f(X) = \mathrm{E}[Y|X]$, so "model" is

$$Y = f(X) + \varepsilon; \qquad \mathrm{E}[\varepsilon|X] = 0$$

Problem: Given a data set of past observations (y_i, x_i), $i = 1, ..., n$

- y_i: observed outcome
- $x_i = (x_{i1}, ..., x_{ip})'$: $p \times 1$ observed predictor vector

produce an estimator $\widehat{f}$ of f to form predictions $\widehat{y} = \widehat{f}(x)$.

So, how should we specify and estimate f?

- Estimation seems "easy": minimize sample analog of forecast loss. E.g.

$$\widehat{f} = \arg\min_{f \in \mathcal{F}} \frac{1}{n} \sum_i (y_i - f(x_i))^2$$

- Specifying $\mathcal{F}$ seems more difficult.

Financial Asset Prediction

Simple illustration to fix concepts: Suppose we want to predict household level net financial assets using income with the conditional mean. (Aside: A conditional median might be preferable.)

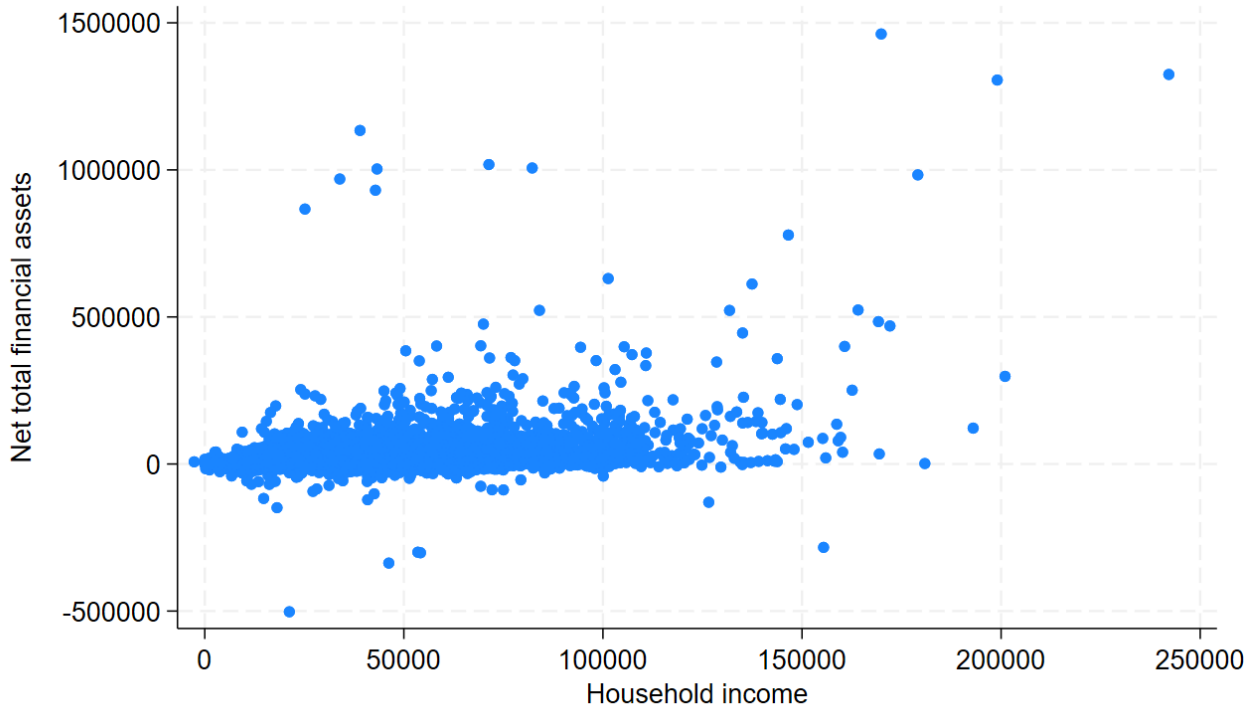
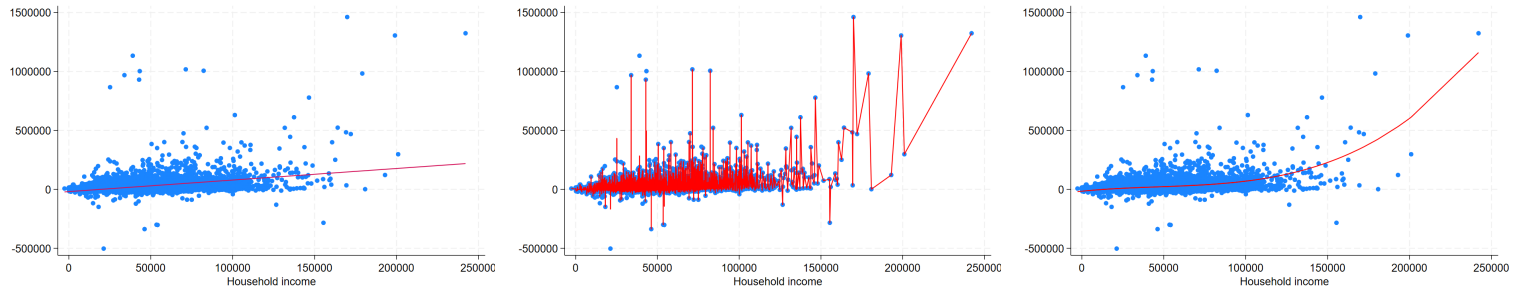


Figure 1: Data from (subset of) SIPP sample for 401(k) example.

Financial Asset Prediction: Candidate Fits

Three candidate prediction rules on the same data:



(a) Linear ($R^2 = 0.161$) **(b)** Kernel ($R^2 = 0.886$) **(c)** Polynomial ($R^2 = 0.244$)

- Linear: Too simple? Missing part of the pattern?
- Kernel: Too complex? Will this generalize?
- Polynomial: About right? Can we do better?

Overfitting/Underfitting

Fundamental problem:

- Find model which is not so simple it misses patterns in data - I.e. avoid underfitting
- Find model which is not so flexible it captures non-generalizable patterns in data - I.e. avoid overfitting

How do we decide?

With one input and moderate numbers of observations, we can use our eyes and intuition. We want something more general.

Prediction Loss

Remember: Goal is to find a rule that does a good job predicting outcome for new observation.

Consider a new/unseen observation: (x_0, y_0) and a prediction rule learned on previous observations $\widehat{f}(\cdot)$

Forecast loss for new observation:

$$MSE = \mathrm{E}[(y_0 - \widehat{f}(x_0))^2]$$

We would like to assess prediction rules on their MSE.

- Could use different losses

The Bias-Variance Trade-Off

Decomposition of the MSE:

$$\begin{aligned} MSE(x_0) &= E(y_0 - \widehat{f}(x_0))^2 \\ &= Var(\widehat{f}(x_0)) + Bias^2(\widehat{f}(x_0)) + Var(\varepsilon_0) \end{aligned}$$

MSE = Variance + Bias squared + non-reducible variance of error

The tension between overfitting and underfitting is often referred to as the "bias/variance tradeoff"

- Bias and variance here refer to bias of the prediction rule under repeated sampling.
- Practical Implication: Prediction rule from your sample is likely to produce systematic mistakes out-of-sample if it has high bias OR variance

In-Sample Fit

Of course, knowing $E\left[\left(y_0 - \widehat{f}(x_0)\right)^2\right]$ is infeasible.

Want a feasible way to estimate quality of fit.

- In-sample analogs offer an obvious solution:

$$\text{(Within-sample) MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \widehat{y}_i)^2 = \frac{1}{n}\sum_{i=1}^{n}(y_i - \widehat{f}(x_i))^2$$

$$\text{(Within-sample) } R^2 = 1 - \frac{\sum_{i=1}^{n}(y_i - \widehat{f}(x_i))^2}{\sum_{i=1}^{n}(y_i - \widehat{f}_0(x_i))^2}$$

- $\widehat{f}_0(\cdot)$ is a baseline prediction rule. R^2 measures improvement of $\widehat{f}$ over this baseline
- Conventional choice: $\widehat{f}_0(x) = \bar{y}$, giving the familiar denominator $\sum_{i=1}^{n}(y_i - \bar{y})^2$

REMEMBER: Goal is "out-of-sample" fit - not "in-sample" fit

- "in-sample fit" $\approx$ "out-of-sample" fit if n is large relative to "model complexity" but in-sample fit not a good guide with complex models
  - standard corrections (e.g. adjusted R^2) are better but also fail with complex models

Out-of-sample fit

Do we really care about explaining what we've already seen?

What matters is accuracy out-of-sample

If we got a bunch of new data NOT USED TO ESTIMATE MODEL (say $\{x_i^o, y_i^o\}_{i=1}^m$), could use this data to evaluate models:

$$\text{(Out-of-Sample) MSE} = \frac{1}{m}\sum_{i=1}^{m}(y_i^o - \widehat{y}_i^o)^2 = \frac{1}{m}\sum_{i=1}^{m}(y_i^o - \widehat{f}(x_i^o))^2$$

$$\text{(Out-of-Sample) } R^2 = 1 - \frac{\sum_{i=1}^{m}(y_i^o - \widehat{f}(x_i^o))^2}{\sum_{i=1}^{m}(y_i^o - \widehat{f}_0(x_i^o))^2}$$

Model Validation

Don't really have data we haven't seen...

Key Idea: Use sample to replicate forecasting environment by splitting data to estimate out-of-sample predictive ability

Split the data:

- Training Sample - data used to estimate prediction rule(s)
- Testing Sample - data used to test estimated rule(s) on NEW data
  - AKA "validation" or "hold-out" sample

Model Validation

Training data: $\{y_i^T, x_i^T\}_{i=1}^{n_T} \rightarrow \widehat{f}_T(\cdot)$

Testing data: $\{y_i^o, x_i^o\}_{i=1}^{m}$

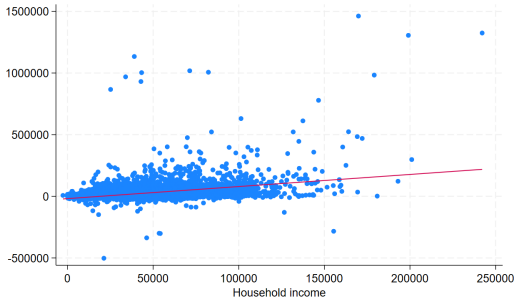
$m + n_T = n$

Estimate of out-of-sample MSE (or R^2) using testing data:

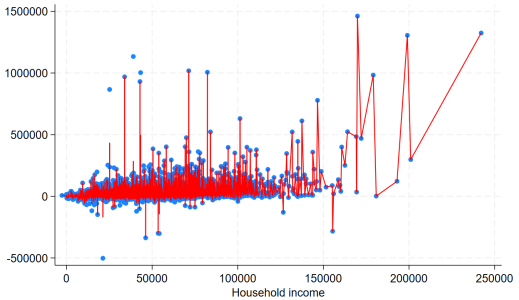
$$\widehat{MSE} = \frac{1}{m}\sum_{i=1}^{m}(y_i^o - \widehat{f}_T(x_i^o))^2$$

$$\widehat{R^2} = 1 - \frac{\widehat{MSE}}{\frac{1}{m}\sum_{i=1}^{m}(y_i^o - \widehat{f}_0(x_i^o))^2}$$

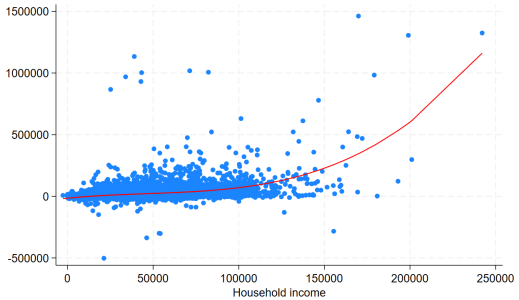
Financial Asset Prediction Revisited



(a) Linear Model Fit



(b) Overfit (Kernel)



(c) Polynomial Fit

Models were fit using $n_T = 7854$ and we kept $m = 2061$ test observations.

In-sample vs out-of-sample fit:

	Linear	Overfit	Polynomial
RMSE (in-sample)	55576	20516	52745
RMSE (out-of-sample)	68706	83167	68959
R2 (in-sample)	0.161	0.886	0.244
R2 (out-of-sample)	0.123	-0.285	0.116

Cross-Validation

Idea of splitting sample and using a training and validation sample good but

1. Depends on choice of training and validation samples
   - Which observations in which sample?
   - How many observations in each?
2. Not using all observations for estimation AND testing

Cross-validation (CV) tries to address these concerns

- Theory for VALIDATION clear and simple
- Theory for cross-validation less transparent

K-Fold CV

1. Divide sample into K approximately equal sized groups
   - n-fold = leave-one-out CV
2. For $k = 1$ to K:
   - Use subsample k as validation sample and use remaining groups as training sample
   - Estimate model using $(Y_{-k}, X_{-k}) \rightarrow \widehat{f}_k(\cdot)$
   - Forecast $\widehat{y}_i = \widehat{f}_k(x_i)$ for all $i \in \mathcal{I}_k$ where $\mathcal{I}_k$ is the set of all indices belonging to group k
3. Estimate generalization error as $\widehat{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \widehat{y}_i)^2$
   - Or use any other prediction criterion as appropriate

Do this for each model/tuning parameter under consideration

Choose the procedure that does best according to forecasting criterion

- Re-estimate selected model using all data
  - theory still being developed: e.g. Chetverikov et al. (2021), Chetverikov and Sørensen (2022)
- Pool predictor $\widehat{f}(X) = \frac{1}{K}\sum_k \widehat{f}_k(X)$
  - general theory providing optimality or near optimality: e.g. Wegkamp (2003), Lecué and Mitchell (2012)

Financial Asset Prediction Revisited

Let's look at 5-Fold CV in our financial asset prediction example:

RMSE by fold:

	Linear	Overfit	Polynomial
Fold 1	63041.474	76649.959	55578.143
Fold 2	44010.977	69666.27	44121.159
Fold 3	41543.506	79414.821	41834.739
Fold 4	60649.763	81475.432	60120.927
Fold 5	64671.224	91462.231	64827.367

Overall CV (R)MSE:

	Linear	Overfit	Polynomial
RMSE	55672.723	80049.04	54042.128

Performance on validation data:

	Linear	KNN	Polynomial
CV Average	0.123	-0.150	0.119
Refit	0.123	-0.285	0.116

Popular ML Methods

Remainder of the lecture will cover some specific methods:

- Penalized regression (Lasso, Ridge)
- Trees, random forests, and boosted trees
- Deep neural networks
  - Transfer learning and pretrained-model fine-tuning
  - Attention and transformers
- Stacking / ensembling
- Interpretation: variable importance, partial dependence, SHAP

Some overview / review papers of ML methods aimed at social scientists:

- Mullainathan and Spiess (2017); Athey and Imbens (2019); Korinek (2023); Brand et al. (2023); Dell (2025)

Penalized Estimation

High-Dimensional Linear Model (HDLM)

Target: build a linear prediction rule for scalar Y from $X = (X_1, ..., X_p)'$ allowing $p > n$

Least squares solves

$$X'X\widehat{\beta} = X'Y$$

When $p > n$ and X is full (row) rank, there are many solutions:

$$\widehat{\beta}^w = (X'X)^- X'Y + [I - (X'X)^- X'X]w$$

for any conformable w, using the Moore-Penrose inverse A^-.

- No unique coefficient estimator
- All solutions fit the sample perfectly: $\widehat{Y} = X\widehat{\beta}^w = Y$ for every w

Simple HDLM Takeaways

When $p/n \not\approx 0$:

- In-sample measures of fit are uninformative
- Parameter estimates unstable/arbitrary
- Linearity alone is insufficient structure
  - Need further **regularization** / dimension reduction
  - Regularization: imposing additional structure to make learning from finite data possible
- Carries over to other GLMs/parameterized models

Motivation for High p

Two key sources of high p:

1. Rich data: data sets increasingly contain many features — aggregate characteristics (country/state/city), price and product characteristics, individual health records, individual tax records, text or image data.

2. Flexible estimation: the best predictor $g(W) = \mathrm{E}[Y|W]$ need not be linear. It can be approximated by a basis expansion

$$g(W) = \sum_{j=1}^{p} \beta_j \varphi_j(W) + r_p(W),$$

with $X = (\varphi_1(W), ..., \varphi_p(W))'$ and approximation error $r_p(W) \searrow 0$ as $p \to \infty$.

- Using approximating functions lets linear prediction approximate $\mathrm{E}[Y|W]$
- When raw input W is moderate-dimensional, p grows fast as flexibility is added

Penalized Estimation

Goal: Learn a prediction rule for Y given X that generalizes out-of-sample in a setting allowing $p \gg n$:

Penalized Estimation

Define penalized estimator

$$\widehat{\beta} = \arg\min_{b \in \mathbb{R}^p} \sum_i \ell(y_i, x_i; b) + \lambda p(b)$$

where

- $\ell(y_i, x_i; b)$ is a measure of forecast loss
- $\lambda p(b)$ is a penalty function that increases in "model complexity"
  - complexity defined in terms coefficients
  - $\lambda p(b)$ key input - form of $p(b)$ determines structure of $\widehat{\beta}$

- Penalized estimator trades off in-sample fit with complexity of the prediction rule
- Penalized GLM's readily available with appropriate choice of loss
  - Can understand estimation results within familiar context of GLM
  - Will only consider $\ell(y_i, x_i; b) = (y_i - x_i' b)^2$ (penalized linear regression)

Canonical Examples

MANY penalized estimators in the literature

- consider two fundamental examples:
  - **Ridge**: $\widehat{\beta} = \arg\min_b \sum_i (y_i - x_i' b)^2 + \lambda \sum_j b_j^2$
  - **Lasso**: $\widehat{\beta} = \arg\min_b \sum_i (y_i - x_i' b)^2 + \lambda \sum_j |b_j|$

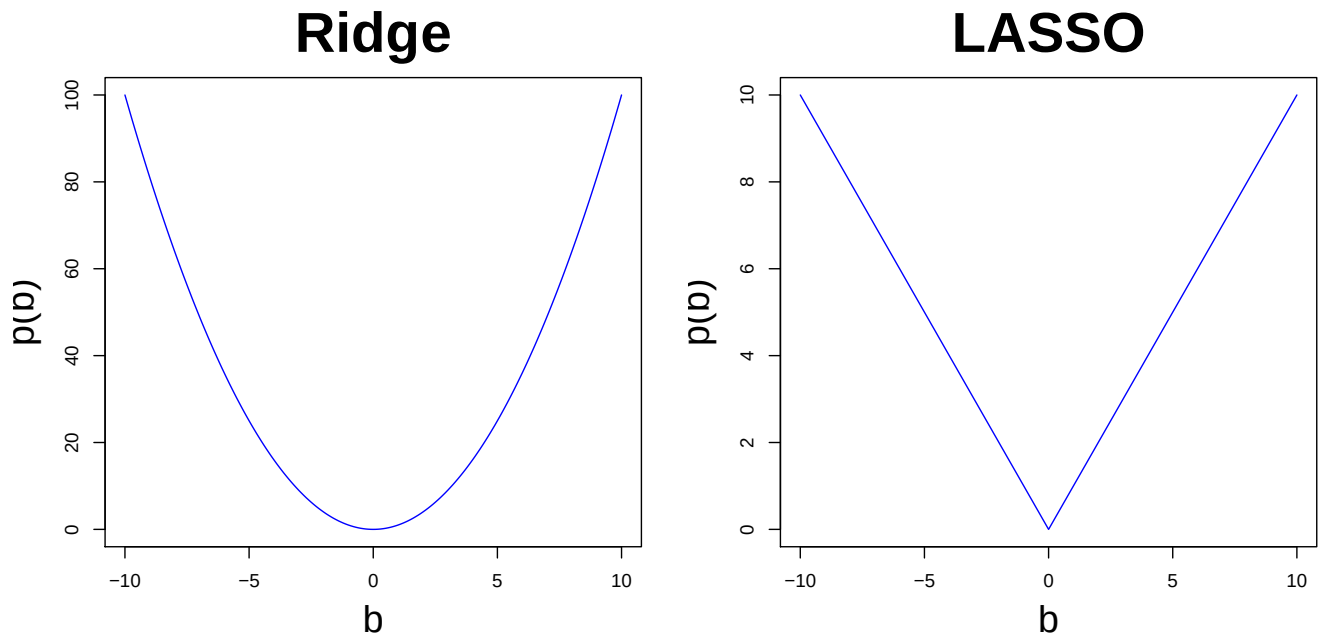


Figure 4: Lasso and Ridge penalty functions in one variable

Ridge vs. Lasso

Ridge: $p(b) = \sum_{j=1}^{p} b_j^2$

- Closed-form solution
- All coefficients non-zero, mostly small — a dense model
- Penalty ≈ 0 for small b_j; very aggressive for large b_j
- **No variable selection**
- Intuitively, dense models are hard to learn — need many observations per parameter (Hsu et al., 2012)

Lasso: $p(b) = \sum_{j=1}^{p} |b_j|$

- Some coefficients set exactly to 0 — a sparse model
- Cost of increasing b_j is constant; kink at 0 forces selection
- **Does variable/model selection**
- Generalizes usual practice of pre-specifying a small set of variables
- Good theoretical properties even when $p \gg n$. "Bet on sparsity" (Friedman et al., 2001); see also Bickel et al. (2009), Belloni and Chernozhukov (2013), Belloni et al. (2012)

Practical Considerations

Choice of λ — tuning parameter controlling shrinkage:

- CV is probably most common; see Chetverikov et al. (2021), Chetverikov and Sørensen (2022) for Lasso theory
- Theoretically valid plug-in choices for Lasso in many settings (heteroscedasticity – Belloni et al., 2012; clustering – Belloni et al., 2016; time series – Chernozhukov et al., 2021); most useful when CV is not obvious (dependence)

Scaling, intercept, loadings:

- Penalty depends on coefficient scale — standardize variables first
  - Canned software standardizes by default; reports results on original scale
- Do not penalize the intercept (unpenalized by default in canned software)
- Easy to allow different penalization of different coefficients, e.g. $p(b) = \sum_j \psi_j |b_j|$ — useful when some variables should be protected from shrinkage, or with non-i.i.d. data (refs above)

401(k) Example: Data

Pick up with the financial asset prediction introduced earlier.

Data:

- 9915 observations from the 1991 SIPP (divided with 7854 training, 2061 validation)
- $y_i =$ net financial assets
- $x_i =$ individual characteristics
  - income, age, family size, education (high school, some college, college), married, two-earner, defined benefit, ira, home-owner, eligibility for a 401(k)
- same data as in Poterba et al. (1995)

401(k) Example: HDLM Predictive Models

We'll consider three different variable structures (number of regressors is number of terms not dropped by `Stata: regress`)

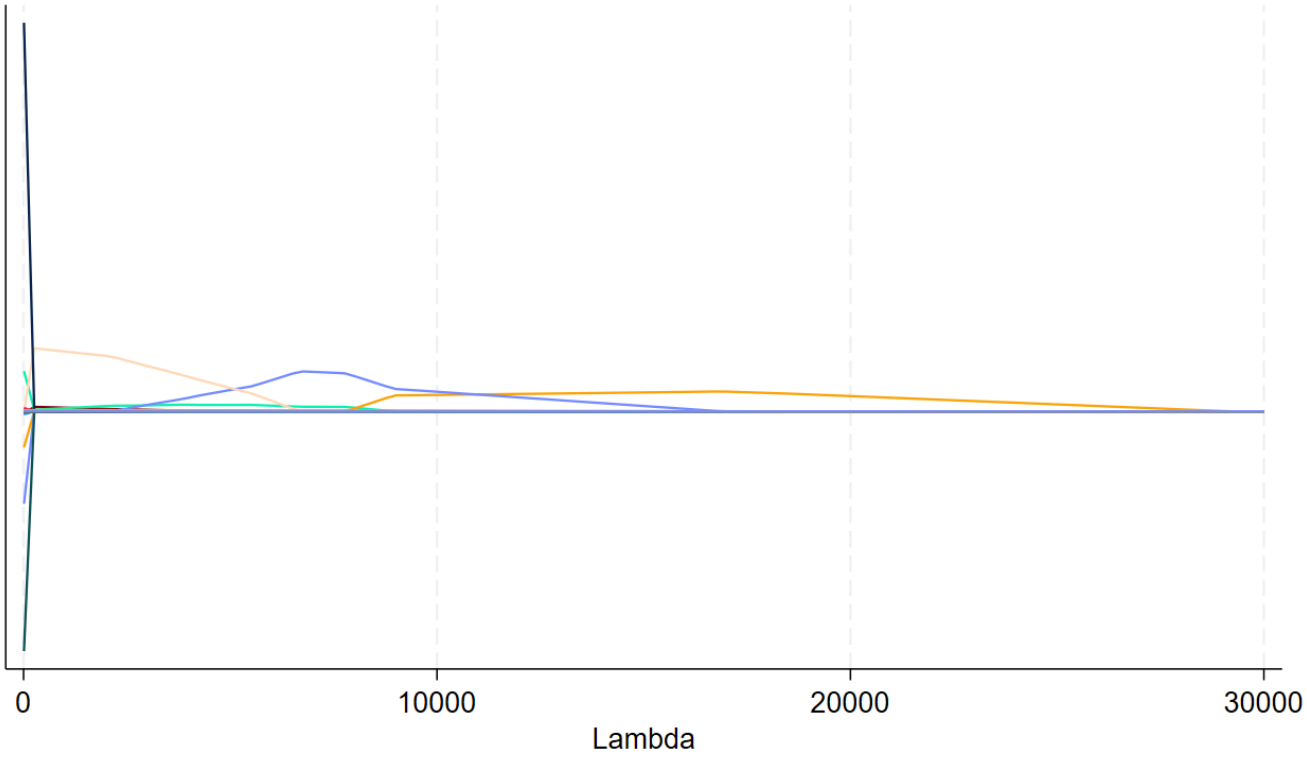
- Basic Model: Raw regressors only
  - p = 10
- Polynomial Model: 6th order polynomial in age, 8th order polynomial in income, fourth order polynomial in education, 2nd order polynomial in family size, + all other variables
  - p = 25
- Interactive Model: Polynomial model + all non-redundant interactions
  - p = 257

We'll estimate each model using OLS and Lasso

- We'll use cross-validation to choose the Lasso tuning parameter
- We'll evaluate performance on a validation sample

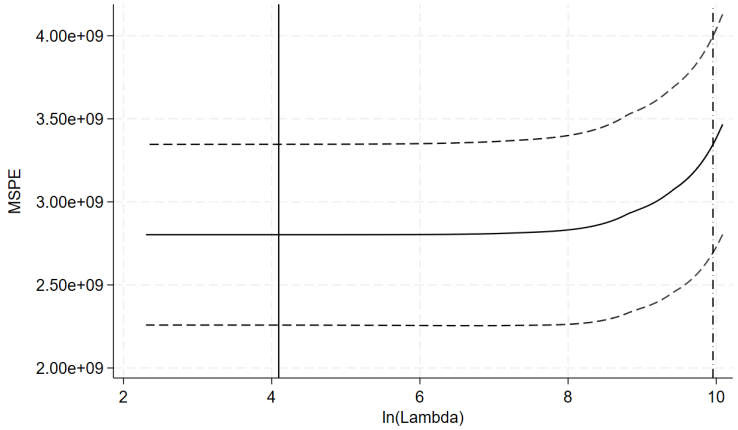
401(k) Example: Lasso Shrinkage

Before turning to actual prediction, let's look at impact of penalty parameter on estimated coefficients in polynomial model

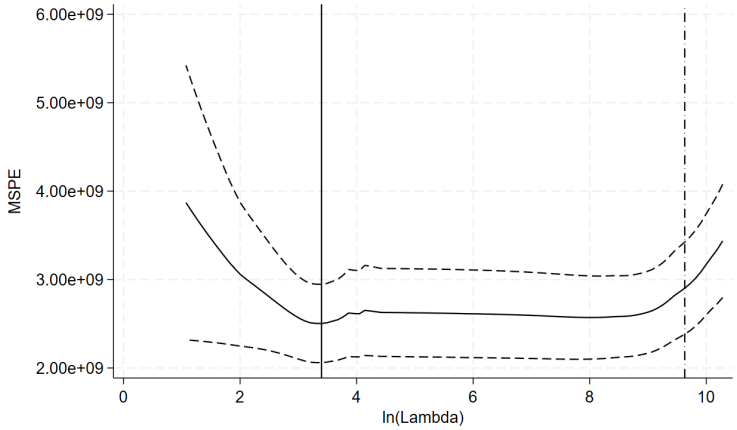


- Lasso is continuous, not smooth in λ
- Zeroes variables out for large enough λ
- Shrinkage does not mean monotonic decrease to 0 as $\lambda \uparrow \infty$

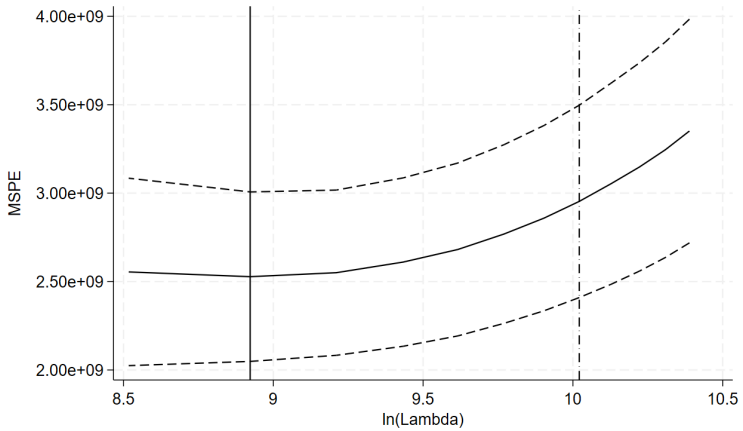
401(k) Example: Cross Validation - Lasso



(a) Lasso CV function for basic design



(b) Lasso CV function for polynomial design



(c) Lasso CV function for interaction design

401(k) Example: CV RMSE and Validation R^2

Minimum cross validated RMSE for each procedure:

	Base	Polynomial	Interactions
OLS	52936	168008	9233934
Lasso	52937	50028	50276

R^2 from the set aside validation data set:

- Model refit using all training data at CV-min tuning parameter value

	Base	Polynomial	Interactions
OLS	0.195	0.181	0.013
Lasso	0.195	0.192	0.177

Lasso is relatively stable across models - OLS not so much

Trees

Tree-Based Methods

Modern nonlinear regression: Aims to obtain a good prediction rule $\widehat{g}(X)$ for $g(X) := \mathrm{E}[Y|X]$ without pre-specifying an approximation for $g(X)$

- Tries to learn how to represent $g(X)$ from the data
- Useful and interesting even when X is low-dimensional
- Examples: trees, deep neural nets

Basic idea of tree-based methods:

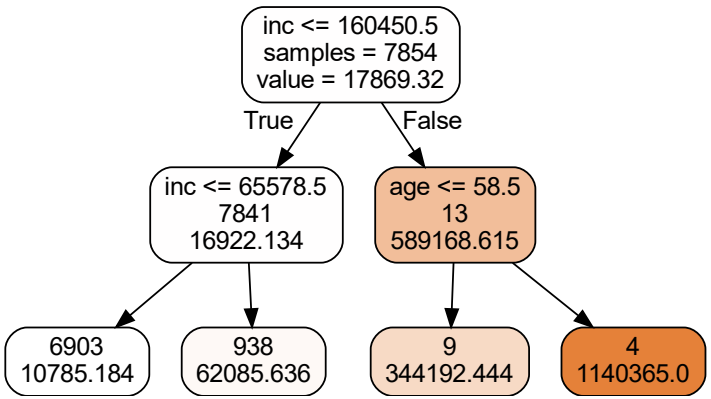
- Adaptively partition the regressor space into regions (usually rectangles)
- Fit a (simple) prediction rule within each region (usually a constant)
- Partition and prediction rule jointly chosen to get good in-sample fit
- With rectangles and constant rule within rectangles: equivalent to approximating $g(X)$ with a step function where steps are *not* pre-specified

Heuristic Regression Tree Algorithm

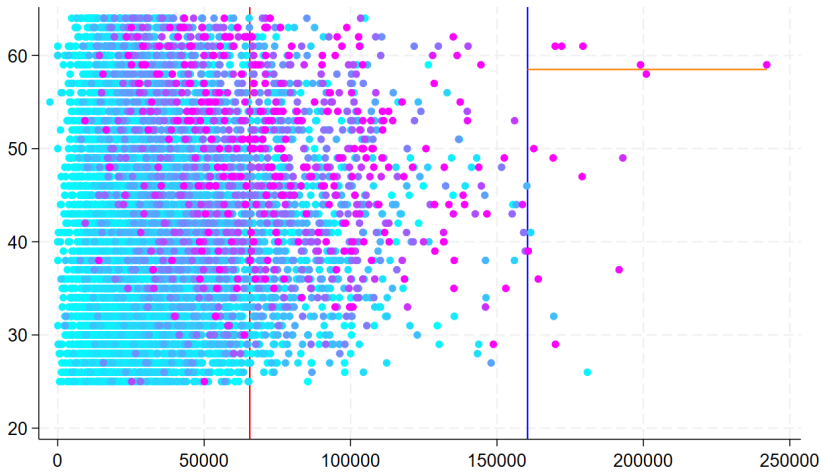
- Basic idea: Use the data to form a decision tree using recursive binary partitions
- Step 1:
  - Let $R_1(j,s) = \{X|X_j < s\}$ and $R_2(j,s) = \{X|X_j \geq s\}$
  - Choose j and s as
    $\arg\min \sum_{i:x_i \in R_1(j,s)} (y_i - \widehat{y}_{i,R_1})^2 + \sum_{i:x_i \in R_{i,2}(j,s)} (y_i - \widehat{y}_{i,R_2})^2$
    - $\widehat{y}_{i,R_k} = \bar{y}_{i,R_k}$ is the forecast of y_i using data in region R_k - just the sample mean of Y in that region
- Step 2:
  - Split the data into the two subregions from Step 1.
  - Repeat the splitting process on each subregion, taking the split that minimizes loss
- etc...
- Stop splitting when some stopping criterion reached (often a minimum node size - e.g. no less than 5 observations - or a maximum number of splits - e.g. no more than 4 terminal nodes)

401(k) Example: Trees

Tree fit in financial asset data from SIPP using only age and income



(a) Tree



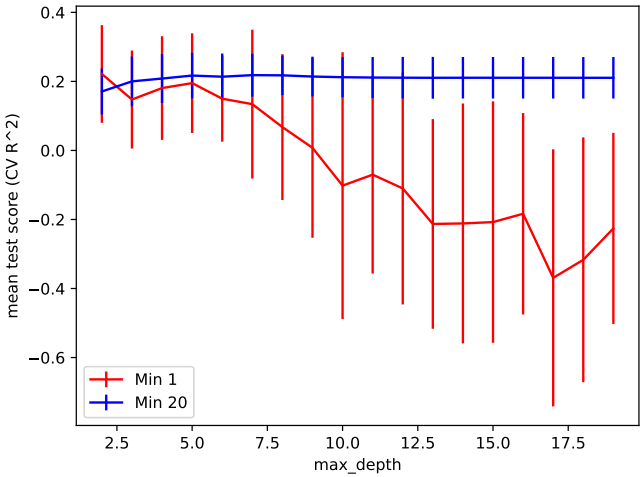
(b) Tree Partition

After three splits – shows both nonlinearity and interaction.

401(k) Example: Prediction Performance

Depth vs. leaf minimum (R^2):

	No Min		Min = 20	
	in-samp	Val	in-samp	Val
Depth 1	0.147	-0.032	0.122	0.113
Depth 2	0.315	0.061	0.215	0.158
Depth 3	0.374	0.106	0.262	0.199
Depth 12	0.880	-0.208	0.350	0.210

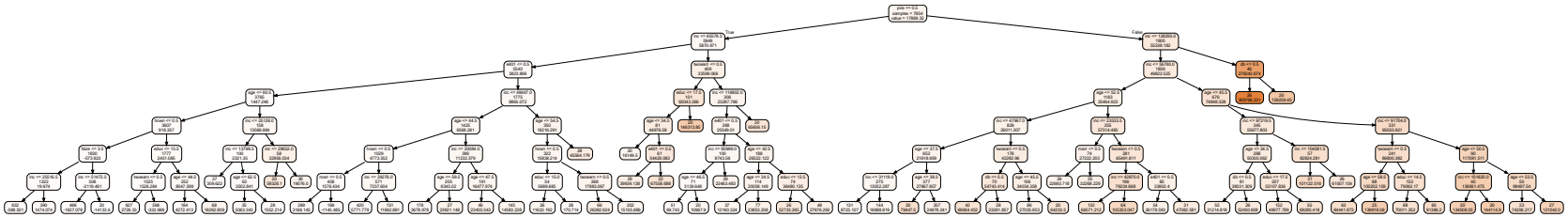


CV R^2 across depth

CV-chosen tree (R^2):

	CV	Validation
Min 1	0.221	0.061
Min 20	0.218	0.223

CV-Chosen Tree and Takeaways



CV-chosen depth-7, Min-20 tree – pretty hard to read (even if blown up).

Good:

- Don't have to think about the scale of *x*'s
- Computationally fast ("scales")
- Small trees are interpretable
- Does automatic interaction detection
- Does variable selection

Bad:

- Step function is crude, may not give the best predictive performance
  - E.g. doesn't do well with linearity
- Big trees are not interpretable

Bagging and Boosting

Trees are often combined with two generic ideas from ML – **Bagging** and **Boosting**

Bagging (Bootstrap Aggregation, Breiman (1996)):

- Basic idea: Fit lots of noisy, low-bias models, then average to reduce variability.
- Use bootstrap (or subsampling) to get training data for fitting the "low-bias models"
- Out-of-bag (OOB) error estimate as byproduct
  - Use observations not included in bootstrap sample to estimate out-of-sample loss
  - OOB MSE $\rightarrow$ leave-one-out CV as number of bootstrap samples $\rightarrow \infty$

Boosting (Schapire (1990), Friedman et al. (2000)):

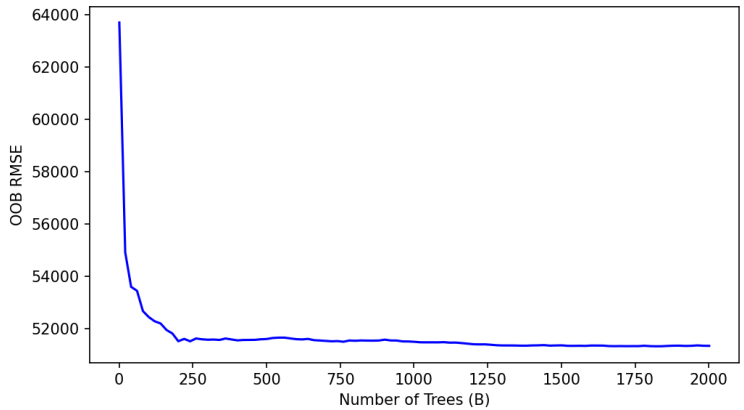
- Basic idea: Improve fit in small increments via recursive fitting on residuals.
- Builds final prediction sequentially by targeting "errors" not captured in previous steps

Random Forests

Applying bagging on top of a tree produces a "random forest":

- For $b = 1$ to B
  - Draw a bootstrap sample $\{y_i^b, x_i^b\}_{i=1}^n$
    - Typical random forest implementation also randomly samples variables. I.e. each x_i^b consists of a (potentially different) random subset of the columns of X
    - Idea is that this "decorrelates" fits across bootstrap samples
  - Estimate model using bootstrap sample $\rightarrow \widehat{f}_b(\cdot)$
- Bagging estimator: $\widehat{f}_{bag}(x) = \frac{1}{B}\sum_{b=1}^{B} \widehat{f}_b(x)$
  - Averaging across step functions with steps chosen at different points
  - Will produce a smooth fit (with continuous X and sufficiently large B)
  - B is a tuning parameter, though results tend to be insensitive once B is "big"
- Sales pitch: Bagged prediction rules less sensitive to tuning choices as long as "low bias" in learning $\widehat{f}_b(\cdot)$
  - use trees with lots of leaves (fast to learn and low bias)
  - tuning still seems to help though (in my experience)

401(k) Example: Random Forests



OOB RMSE vs. B:

- Stable for $B \gtrsim 250$
- Default model gives validation $R^2 \approx 0.21$

Tuning-choice comparison:

	OOB RMSE	Validation R^2
Default	51487	0.211
No X Randomization	53388	0.151
Min Size(20)	52057	0.224
Min Size(40)	52696	0.219
Min Size(80)	53431	0.202
Min Size(160)	54128	0.179

- Relatively stable across tuning choices – except turning off X randomization
- OOB $\neq$ CV: use comparable metrics when comparing across methods

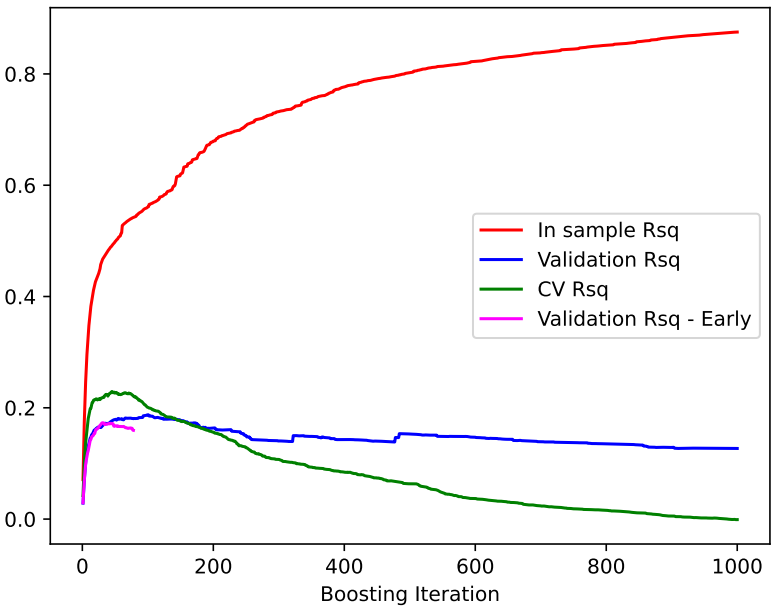
Boosted Trees

Basic algorithm for applying boosting using trees as the "base learner" with data $\{y_i, x_i\}_{i=1}^n$:

1. Initialize residuals $r_i = y_i$
2. For $b = 1, \ldots, B$:
    - Fit a simple tree-based rule $\widehat{g}_b(X)$ on $\{r_i, x_i\}$
        - Typically a shallow tree of depth d; $d = 1$ gives an additive nonparametric model, $d = 2$ allows pairwise interactions, ...
    - Update residuals $r_i \leftarrow r_i - \lambda \widehat{g}_b(x_i)$; $\lambda \ll 1$ is the "learning rate"
3. Final rule: $\widehat{g}(X) = \sum_{b=1}^B \lambda \widehat{g}_b(X)$

Tuning parameters (B, λ, d) chosen by cross-validation. Strong potential to overfit if B is too large.

401(k) Example: Iterations and Early Stopping



In-sample, validation, and CV R^2 against B (default tuning):

- In-sample keeps climbing as $B \nearrow$; validation peaks and then falls $\Rightarrow$ overfitting
- Val $R^2 \approx 0.187$ at the best B (in validation)
- Val $R^2 \approx 0.176$ at CV-chosen B

Early stopping: monitor a held-out fraction during training, halt once it stops improving.

- Tolerance 10^{-4}, 10 iterations of no improvement $\Rightarrow$ stops at $B = 78$
- Early-stopping Val $R^2 \approx 0.159$

401(k) Example: Other Tuning Choices

All impose minimum leaf size of 20

Depth	Rate	Iter	CV RMSE	in-sample R^2	Validation R^2
6	0.3	500	59020.036	0.823	0.058
6	0.3	e.s.	52541.337	0.405	0.205
6	0.1	500	54218.371	0.638	0.125
6	0.1	e.s.	52552.372	0.397	0.205
5	0.1	e.s.	52344.737	0.374	0.205
4	0.1	e.s.	52369.538	0.365	0.202
3	0.1	e.s.	52163.745	0.322	0.187
2	0.1	e.s.	52708.711	0.331	0.179
6	0.01	e.s.	53467.074	0.276	0.173
5	0.01	e.s.	53631.916	0.263	0.167
4	0.01	e.s.	53819.876	0.246	0.159
3	0.01	e.s.	54197.389	0.228	0.148
2	0.01	e.s.	55020.508	0.199	0.132

Deep Neural Networks

Single Layer Neural Networks

A single layer neural network (NN) produces a prediction rule of the form:

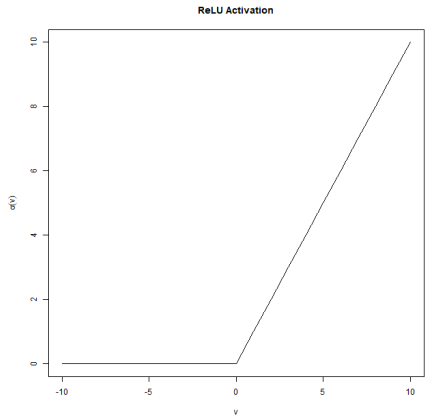
$$\widehat{g}(x) = \sum_{m=1}^{M} \widehat{\beta}_m Z_m(\widehat{\alpha}_m)$$

- $Z_m(\widehat{\alpha}_m)$ are constructed regressors called **neurons**
  - Always set $Z_1(\alpha_1) = 1$
  - For $m = 2, ..., M$, $Z_m(\alpha_m) = \sigma(\alpha_m' X)$ for **activation function** $\sigma(\cdot)$
- a NN is a (generalized) linear combination of "learned" predictor variables $\{Z_m(\widehat{\alpha}_m)\}_{m=1}^{M}$
  - to be interesting, $\sigma(\cdot)$ should be nonlinear
  - **Representation Learning**: rather than hand-engineering features, we learn **embeddings**—new representations of the raw inputs designed to make predicting Y easier.
- estimated by empirical loss minimization with a variety of regularization schemes (more on this later)

Activation functions

Most popular activation function (currently) is the **rectified linear unit** function (ReLU):

$$\sigma(v) = \max(0, v)$$



Other common activation functions:

- softplus (smoothed ReLU): $\sigma(v) = \log(1 + \exp(v))$
- sigmoid: $\sigma(v) = \frac{1}{1+\exp(-v)}$
- modern nets often use smooth ReLU-like variants
  - GELU: Gaussian Error Linear Unit
  - SiLU/Swish: Sigmoid Linear Unit
  - These are now standard in transformers

Deep Neural Networks

Deep Neural Networks (DNN; aka Deep Nets, Deep Learning) adds more **layers** to the NN structure

- The DNN structure is repeated composition of nonlinear mappings

$$X \overset{f_1}{\longmapsto} H^{(1)} \overset{f_2}{\longmapsto} \quad ... \overset{f_m}{\longmapsto} H^{(m)} \overset{f_{m+1}}{\longmapsto} Z$$

- **neurons**: $H^{(\ell)} = \{H_k^{(\ell)}\}_{k=1}^{K_\ell}$
- Mapping from one layer to next:

$$f_\ell : v \longmapsto \{H_k^{(\ell)}(v)\}_{k=1}^{K_\ell} := (1, \{\sigma_{k,\ell}(v'\alpha_{k,\ell})\}_{k=2}^{K_\ell})$$

- **Input Layer**: X (raw predictor variables)
- **Hidden Layers**: Layers between input layer and output
- Final prediction model uses Z as inputs (e.g. $\widehat{g}(X) = \beta' Z$)
- Let η collect all the $\alpha_{k,\ell}$'s and parameters of the final prediction model

Training a DNN

"Train" DNN by attempting to solve

$$\min_{\eta} \sum_{i} (y_i - \widehat{g}(X; \eta))^2 + \text{penalty}(\eta, \lambda) \tag{4.1}$$

- Could use other loss
- Penalty function (e.g. Ridge style, Lasso style) with penalty parameter(s) λ
- Generally non-convex and highly over-parameterized
- Lots of tuning parameters: depth, width, connectivity, explicit penalization, dropout, early stopping, optimization settings, ...
- No great theoretical guidance — *ad hoc* choice and (cross-)validation

DNN explosion driven by advances in computation, optimization, and structure — e.g. Goodfellow et al. (2016). We'll briefly cover

- Stochastic gradient descent (SGD)
- Dropout
- Network depth and width

playground.tensorflow.org

Before turning to some details, which play with some simple NN's in a nice visual structure available at playground.tensorflow.org

Stochastic Gradient Descent (SGD)

Basic gradient descent:

- Inputs: initial condition η_0, learning rate ν, gradient $\nabla f(\eta)$
- Iterate until convergence: $\eta_t = \eta_{t-1} - \nu \nabla f(\eta_{t-1})$

SGD is gradient descent where the gradient is computed using subsets of observations

- **batch**: subsample of data (often single observations)
- **epoch**: a single cycle through all observations

Advantages of SGD:

- fast and scalable (only need little bits of data at a time)
- noise in gradient computation has some advantages — e.g. helps avoid saddle points
- SGD tuning parameters (epochs, learning rate, batch size) offer another lever for regularization

In practice, typically use adaptive variants of SGD (e.g. Adam)

Dropout

Dropout regularization:

- randomly drop neurons during each training step
- tuning parameter p - probability of dropping a neuron
  - can use different probabilities for different layers
  - common default choice is $p = .5$ - though typically tuned
- regularizes by reducing "co-adaptation" of neurons
  - E.g. pretend one neuron captured the generalizable prediction pattern. Given this other neurons can specialize (co-adapt) to fit idiosyncrasies in the training data.
  - Dropout encourages more robust (i.e. regularized) networks by not "allowing" this type of overspecialization

Network Depth and Width

Good approximation of a target function via a DNN can be achieved with sufficient depth/width (not surprising). E.g., Yarotsky (2017); Schmidt-Hieber (2020); Farrell et al. (2021); Kidger and Lyons (2020)

- Depth requires more sequential computations — less easy to parallelize
- Lu et al. (2017) suggest depth may be more important than width for ReLU networks
- Mhaskar et al. (2017): compositional functions approximated by shallow or deep networks, but deep networks require (exponentially) fewer parameters
- **Residual (skip) connections**: allow information from an earlier layer to skip ahead and be added to a later layer
  - Instead of forcing each layer to completely rewrite the representation, the network can keep the old representation and learn an adjustment
  - Schematic notation: $H^{(\ell)} = H^{(\ell-1)} + f_\ell(H^{(\ell-1)})$, so the layer learns an update to the previous representation rather than replacing it entirely
  - These skip paths help signals move through very deep networks, making depth easier to train
  - Key ingredient in modern architectures, including ResNets and transformers

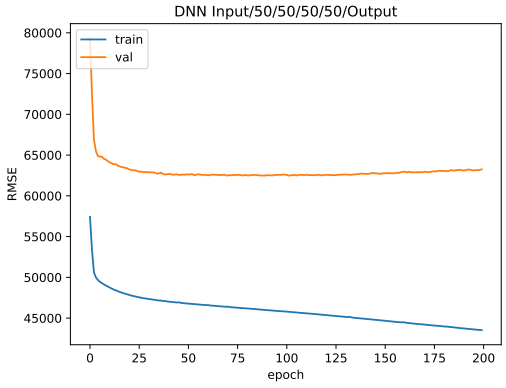
401(k) Example: Neural Nets

Revisit 401(k) data. Basic structure shared across all four models:

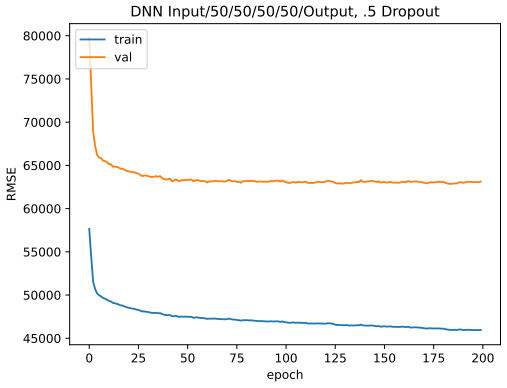
- Input layer $\mapsto$ 50 neurons $\mapsto$ 50 neurons $\mapsto$ 50 neurons $\mapsto$ 50 neurons $\mapsto$ output layer, fully connected
- Trained with batch size 200, 20% hold-out sample
- Vary only the regularization

Model	Regularization	Validation R^2
A	none (200 epochs)	0.205
B	dropout rate 0.5 (200 epochs)	0.226
C	ℓ_2 penalty (200 epochs)	0.205
D	early stopping	0.238

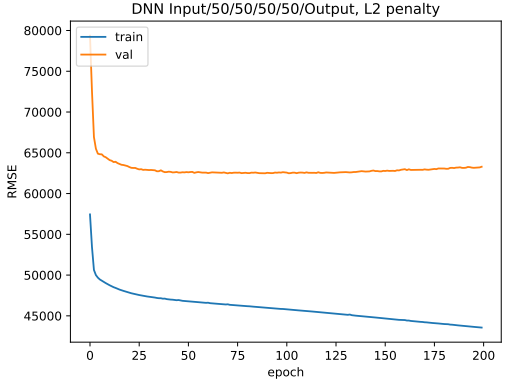
401(k) Example: Training Curves



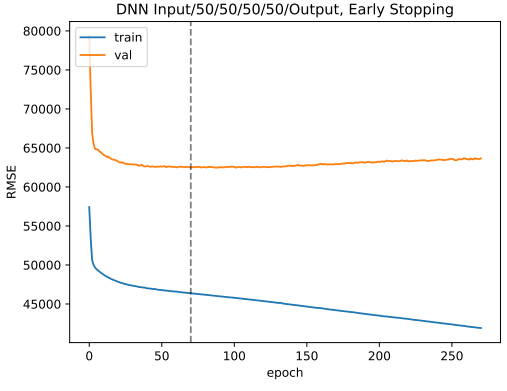
(A) no regularization



(B) dropout



(C) ℓ_2 penalty



(D) early stopping (Exactly as described for boosting.)

401(k) Example: Other Tuning Choices

Structure	ℓ_2/e.s.	RMSE (in)	RMSE (out)	Validation R^2
20/20	0	55642	68122	0.138
20/20	0.1	55569	68049	0.140
20/20	1	55556	68034	0.140
20/20	e.s.	51188	64411	0.229
50/10/50	0	50938	64268	0.233
50/10/50	0.1	50958	64258	0.233
50/10/50	1	50957	64256	0.233
50/10/50	e.s.	46849	65553	0.202
20	0	61517	74145	-0.021
20	0.1	61511	74140	-0.021
20	1	61511	74140	-0.021
20	e.s.	57129	69728	0.097

Many more knobs available (depth/width, dropout rates, optimizer settings, connectivity patterns, . . .).

Transfer Learning

Transfer Learning

DNNs are useful in many cases, but training from scratch is challenging:

- Want *lots* of data for most tasks
- Want specialized computing infrastructure (GPUs, TPUs, . . .)
- Heavy experimentation over architecture and other design choices

Many powerful pre-trained models are publicly available

- Hugging Face hub: huggingface.co
- PyTorch / torchvision model zoos
- Pre-trained models can generalize beyond their original tasks — learned representations are often useful in related but different problems

Transfer learning: start from a pre-trained network and adapt it to a new task.

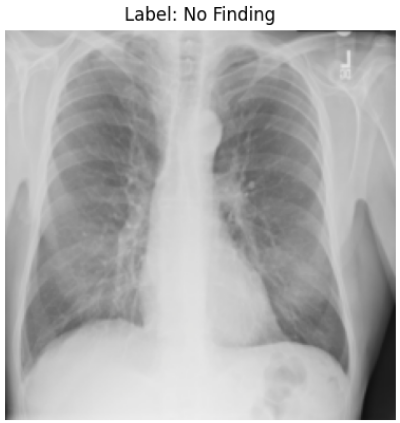
Transfer Learning: How

Two common modes (and combinations of them):

- **Feature extractor**: freeze the pre-trained body, attach a small new head, train only the head
  - Few trainable parameters $\Rightarrow$ low data requirement, fast
  - Implicit assumption: pre-trained representations already useful
- **Fine-tuning**: continue training some or all parameters from the pre-trained initialization, typically with a small learning rate
  - More flexible $\Rightarrow$ adapts representations to the new task
  - More parameters trainable $\Rightarrow$ needs more data, more compute, more care to avoid overfitting

Often add new task-specific layers (e.g. swap the original 1000-class ImageNet head for a binary classifier).

Example: Identifying X-Ray Anomalies



Label: No Finding

Label: Infiltration

Task: chest X-ray binary classification, “No Finding” vs. “Finding”.

- Source: NIH Chest X-ray Dataset
- Subset: 8000 images; 5600 training / 2400 validation
- Reading X-rays takes years of training — a real test of whether ImageNet features transfer

Base Model: ResNet-18

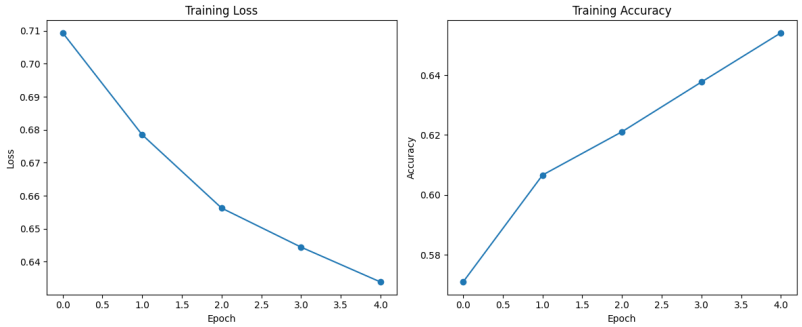
Start from **ResNet-18** (He et al., 2016):

- 18-layer deep residual network, $\approx$ 11.7 million parameters
- Pre-trained on ImageNet ($\approx$ 1.2 million labeled images, 1000 categories)
- Lightweight enough to fine-tune on a laptop (no GPU required)

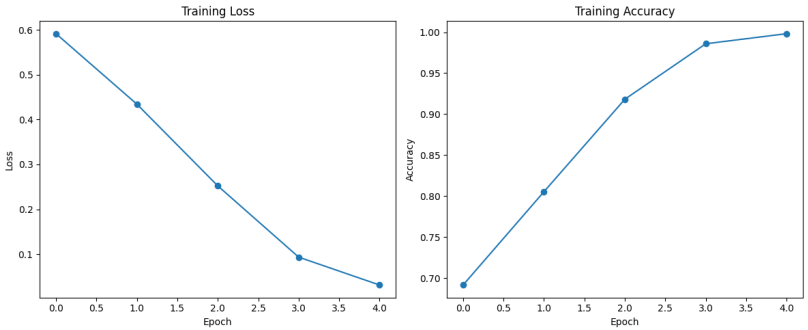
Will it transfer?

- Both source (ImageNet) and target (X-rays) are images $\Rightarrow$ low-level features (edges, textures, contrast) plausibly carry over
- ImageNet contains everyday objects (cats, dogs, . . .), *not* medical scans

Training: Last Layer Only vs. All Layers



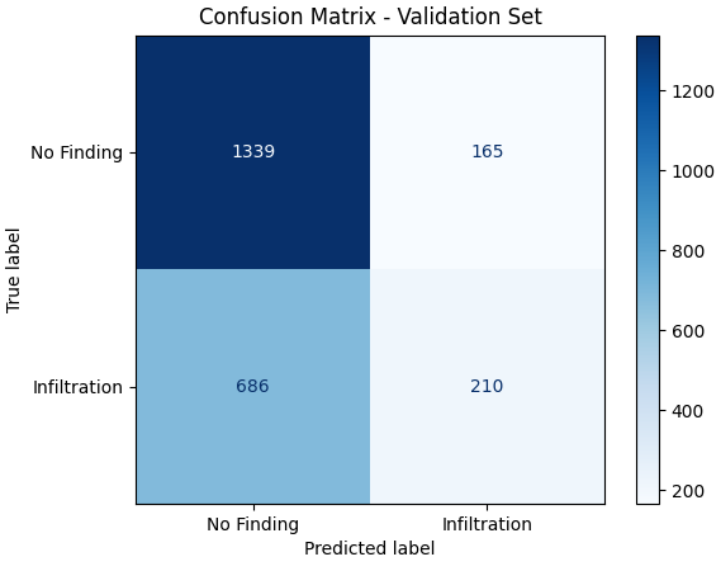
Last layer only (1026 trainable parameters)



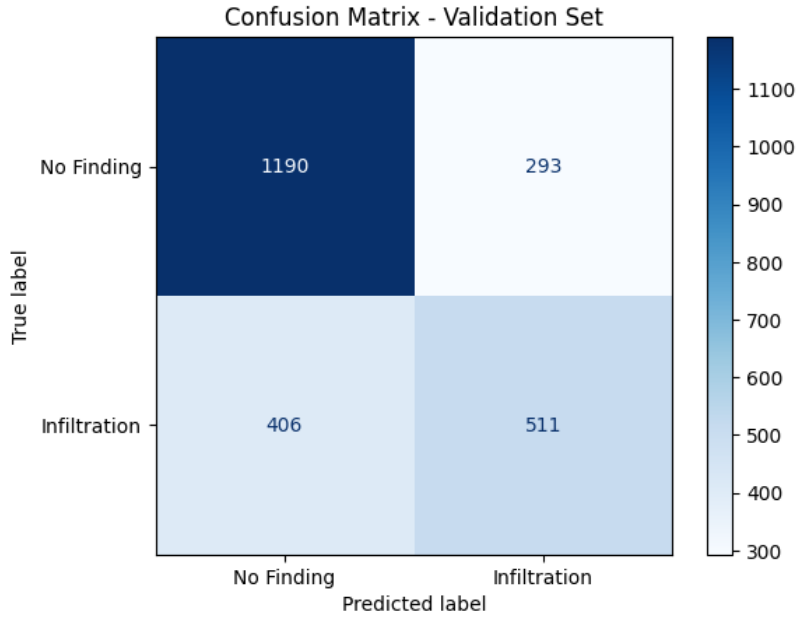
All layers free (11,177,538 trainable parameters)

- Trained 5 epochs each, Adam, learning rate 3×10^{-5}
- All-layers-free fits the training data far more aggressively, as expected

Validation Performance



Last layer only — accuracy 0.645



All layers free — accuracy 0.709

- Majority-class baseline accuracy ≈ 0.62
- Feature-extractor mode barely beats the baseline; fine-tuning all layers seems meaningfully better
- Especially helps on the minority “Infiltration” class: recall jumps from 0.23 to 0.56
- Note that result may vary from run to run

Attention & Transformers

Tokens & Embeddings

Modern text models don't "see" words — they see **tokens**: integer IDs for learned subword units (words, character clusters, punctuation, whitespace).

E.g. "Prof. Hansen teaches machine learning to MBA students."

- Token IDs: 19117, 13, 88786, 45459, 7342, 7524, 316, 53298, 4501, 13
- Token strings: `'Prof', '.', ' Hansen', ' teaches', ' machine', ' learning', ' to', ' MBA', ' students', '.'`

Integer IDs are arbitrary, so map each token to a vector in $\mathbb{R}^d$:

- **Embedding**: a learned map from token ID to a vector, trained jointly with the rest of the model
- Typical $d \approx$ 1,000 to 10,000
- Vector space supports addition and distance — in particular, cosine similarity (think correlation between two vectors)
- Idea of tokenization and embedding extends beyond text

Geometry of Meaning

Classic demo. Take embeddings for `good`, `bad`, `happy`, `sad`. Compute cosine similarity to v_{good}:

	v_{bad}	v_{happy}	v_{sad}	$v_{\text{good}} + (v_{\text{sad}} - v_{\text{bad}})$
Cosine sim. to v_{good}	0.612	0.662	0.445	0.694

- Synthetic point (don't take too seriously)
  - "sad" is a "bad" feeling: "sad - bad" $\approx$ "feeling"
  - "good" + "feeling" $\approx$ "happy"
- Embeddings appear to capture something about the "geometry of meaning"

Contextual Embeddings

A given token's vector should depend on its *context*: "bank" in "investment bank" is not the same as "bank" in "river bank."

Table report cosine similarity between Phrase 1 and Phrase 2 using

(1) average the static token vectors ("BoW-avg")

(2) a model that returns a single context-aware vector ("contextual")

Phrase 1	Phrase 2	BoW-avg	Contextual
bank	river bank	0.843	0.432
bank	investment bank	0.843	0.614
bank	piggy bank	0.825	0.526
bank	sat on the river bank	0.691	0.313
river bank	investment bank	0.743	0.386
river bank	sat on the river bank	0.802	0.619
investment bank	piggy bank	0.751	0.493

A token's static embedding alone can't encode context-dependent meaning

Words in Context: Why Attention

- X: matrix of token vectors
- Query ($Q = XW^Q$), Key ($K = XW^K$), and Value ($V = XW^V$)
  - Just three rotated and scaled copies of X
  - W^Q, W^K, W^V matrices of parameters to be learned
  - R is key/query dimension (which can differ from value dimension)

Attention (Vaswani et al. (2017)):

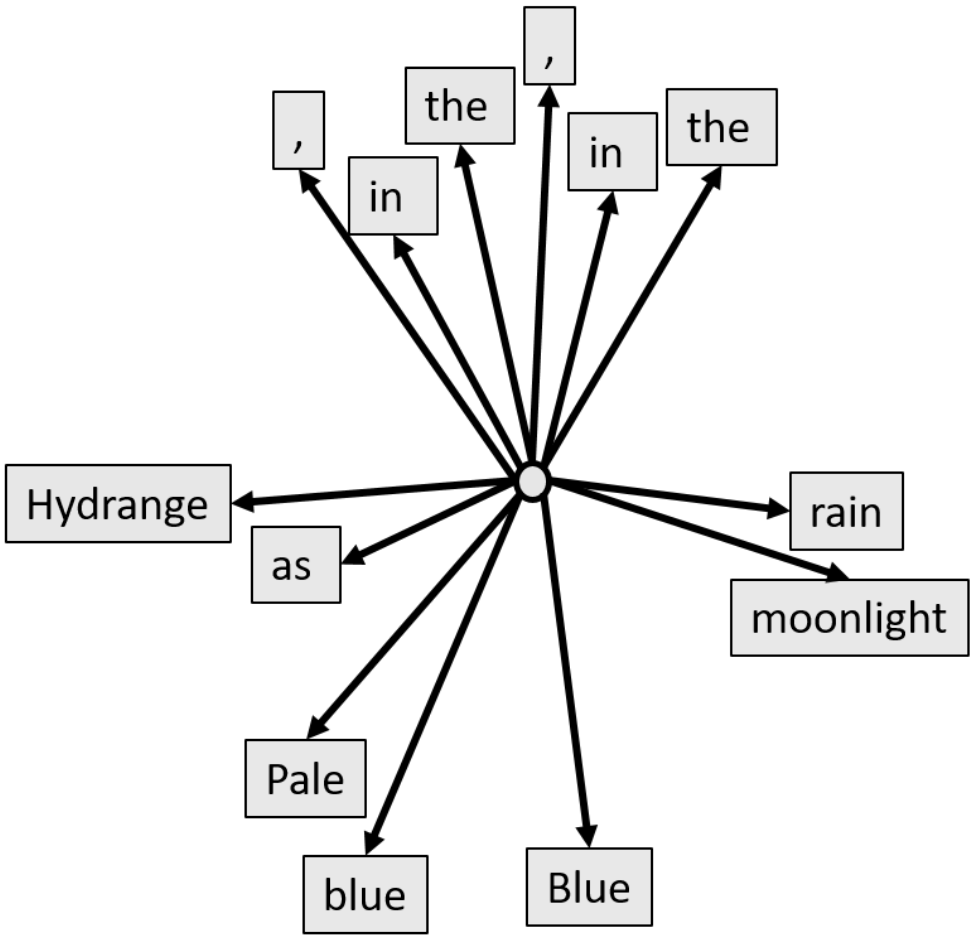
$$Z = \text{softmax}\left(\frac{QK^\top}{\sqrt{R}}\right) V$$

Z (output of self-attention layer) gives new representation for each token by "paying attention" (putting high weight) on other tokens that influence its meaning

The next few frames unpack what this is doing.

Visualizing Attention: a Stylized 2D Embedding

"Hydrangeas, Pale blue in the rain, Blue in the moonlight."
(By Masaoka Shiki — Translation by Lucien Stryk)



Recall that we are representing tokens as vectors.

Here we represent each token as a 2-dimensional vector.

In this stylized embedding:

- Location in the sentence is encoded with earlier positions appearing closer to the left.
- Nouns appear near the horizon.
- Descriptors appear below the horizon.
- Grammatical tokens appear above the horizon.

Make Query & Key Copies, then Align Them

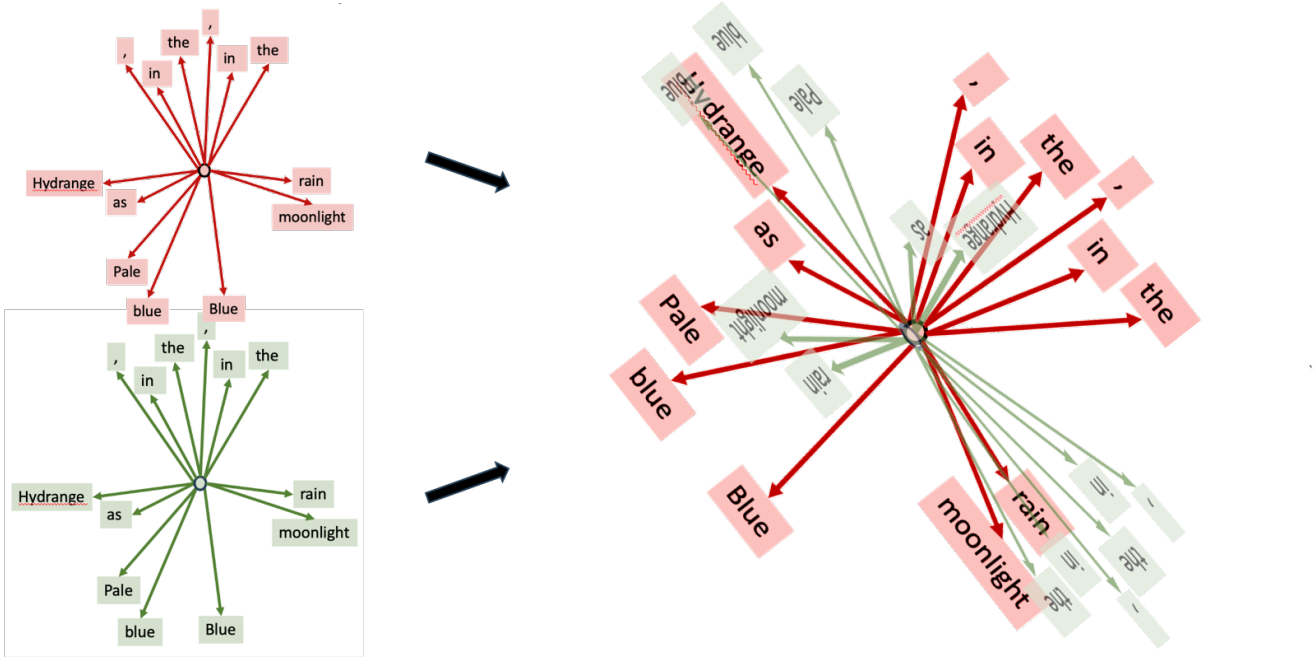
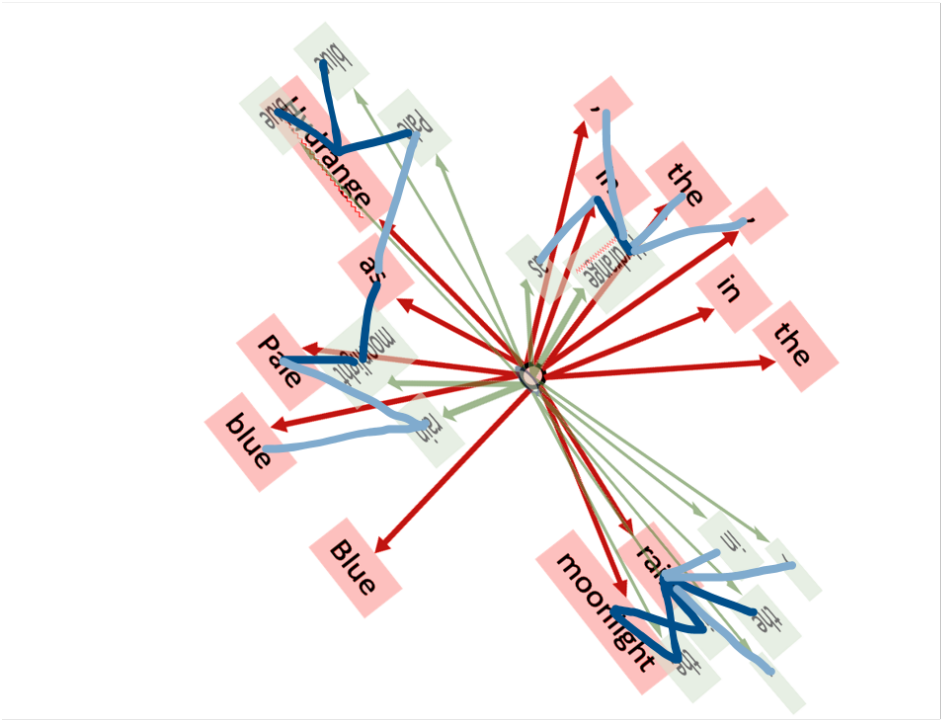
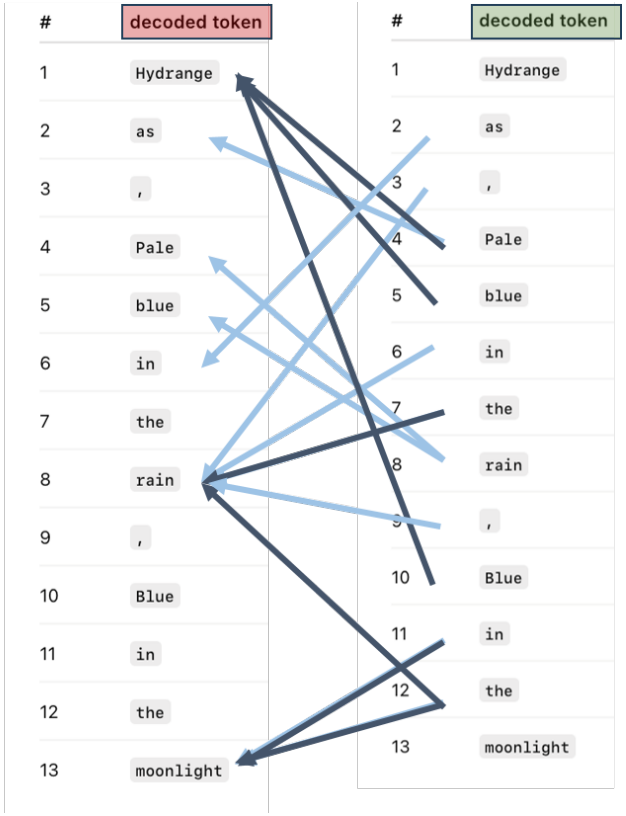


Figure 11: First duplicate to query (red) + key (green) copies. Then rotate / scale the copies with W^Q, W^K.

The learned matrices W^Q, W^K realign the query and key spaces so that "query *i* matches key *j*" captures whatever notion of similarity is useful for the task.

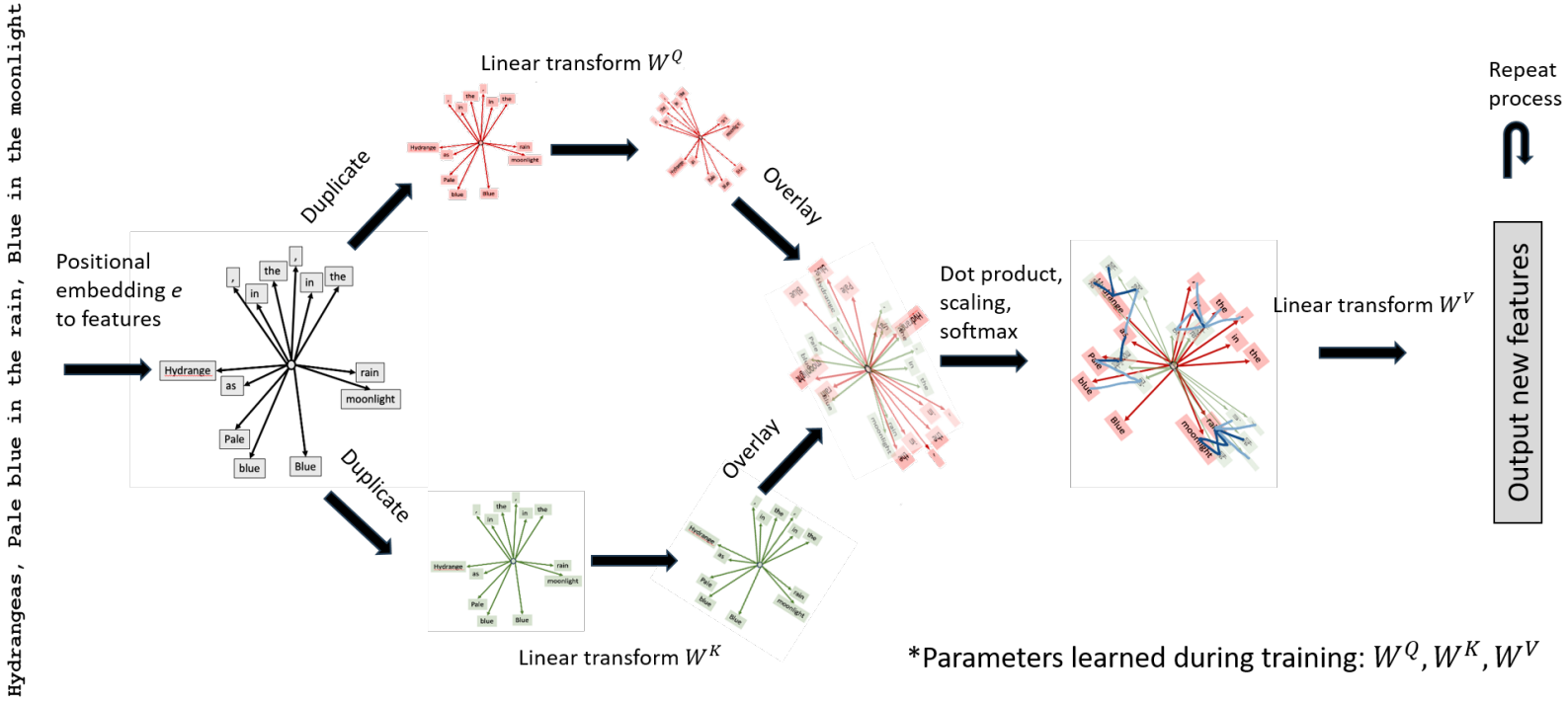
Attention Measures Query–Key Overlap





- Dark blue indicates “large query-key similarity”
- Light blue indicates “moderate query-key similarity”
- After initial attention scores are computed, they are scaled and transformed by applying a softmax.

Putting It All Together



Each token's new representation is a weighted combination of *value* copies of all tokens, where weights come from query–key overlaps.

Multi-Head Attention, Transformers, and Training

Multi-head attention: run several attention mechanisms in parallel, each with its own W^Q, W^K, W^V. Different heads can learn to capture different aspects of context (grammar, position, topic, . . .).

Transformer: a DNN architecture that stacks (multi-head) attention layers with feedforward layers, residual connections, and normalization.

Training: predict the missing token.

- Mask a token in raw text and train the model to fill it in: “Hydrangeas, [MASK] blue in the rain” $\rightarrow$ `Pale`
- Manufactures lots of training data from any text corpus
- The masking trick generalizes — different mask patterns enable different uses (next-token generation, in-context / few-shot prompting, instruction tuning)

Stacking: Leveraging Strength of Many Learners

Stacking

Introduced several methods for building predictions, all of which rely on additional tuning parameters

- Hard to know what will work best in any given setting
- rather than choose a single approach, combine several

Stacking: a linear combination of prediction rules; intuitively can't do worse than picking the single best model.

Stacking objective function using K-fold CV:

$$\min_{w_1,\ldots,w_J} \sum_{i}^{n} \left(y_i - \sum_{j=1}^{J} w_j \widehat{f}_j^{\,k(i)}(z_i) \right)^2, \qquad \text{s.t. } w_m \geq 0, \ \sum_m |w_m| = 1$$

where $\widehat{f}_j^{\,k(i)}(z_i)$ are cross-validated predicted values

- Constraints traditional but not required (drop with few learners)
- Can also use other penalization (e.g. Lasso, Ridge) on the weights

401(k) Example: Candidate Learners

Apply stacking to combine ten candidate learners on the 401(k) asset data:

1. OLS - basic
2. OLS - interactive
3. Lasso (CV) - interactive
4. Ridge (CV) - interactive
5. Random forest
6. Boosted trees - depth 5
7. Boosted trees - depth 3
8. DNN - 50/10/50
9. DNN - 50/50/50/50
10. DNN - 100/100/100/100/100

401(k) Example: Stacking Results

	In R^2	CV R^2	Test R^2	Stacking Weight
OLS - basic	0.245	0.238	0.195	0.000
OLS - flexible	0.513	-85111	0.087	0.001
Lasso (CV)	0.386	0.176	0.200	0.000
Ridge (CV)	0.351	-0.130	0.195	0.224
Random forest	0.345	0.256	0.223	0.000
Boost - depth 5	0.374	0.256	0.205	0.000
Boost - depth 3	0.322	0.261	0.187	0.045
DNN - 50/10/50	0.295	0.273	0.233	0.280
DNN - 50/50/50/50	0.420	0.288	0.188	0.000
DNN - 100/100/100/100/100	0.450	0.320	0.175	0.496
Stacking	0.420	0.420	0.213	-

Black Box Model Interpretation

Black-box models

Random forests, boosted trees, neural nets are effectively “black-box” prediction algorithms.

- produce forecasts but it’s hard to really see what goes into the prediction rule
- many (all?) flexible/nonparametric procedures have this feature
  - big trees hard to think about
  - parameterized models with many interactions and nonlinear terms hard to think about

One sensible option is to view such rules as algorithms to make predictions and not try to (over-)interpret the results

- Good enough for many uses in the social sciences and industry

Variable Importance

Sometimes one wants to dig a bit deeper into a “black-box” algorithm

- Variable Importance measures aim to assess how different variables contribute to a model’s predictions

Several measures out there:

- *Drop-column importance* – refit the model leaving out X_j and compare losses; conceptually clean but expensive (need to refit p times)
- *Permutation importance* – shuffle X_j and re-evaluate the trained rule’s loss (Breiman, 2001); cheap but tied to a single fit
- *Shapley/SHAP values* – attribute the prediction itself to each variable using a cooperative-game-theory decomposition (Lundberg and Lee, 2017)

We focus on SHAP for the running example: it gives both a global importance ranking *and* per-observation attributions.

SHAP Values: Concept

SHAP (SHapley Additive exPlanations) adapts an idea from cooperative game theory (Lundberg and Lee, 2017).

Cooperative-game intuition (Shapley 1953):

- A coalition of players generates a payoff
- Want to split the payoff among players in proportion to each one's marginal contribution – averaged over all orderings in which they could join the coalition

In our setting:

- Players $\Leftrightarrow$ input variables $X_1, \ldots, X_p$
- Payoff $\Leftrightarrow$ the model's prediction $\widehat{g}(x)$
- SHAP value $\phi_j(x)$ = variable j's share of the prediction at observation x

SHAP Values: Definition

Let $S \subseteq \{1, \ldots, p\}$ index a subset of variables and define the coalition value

$$v(S) = \mathbb{E}_n[\,\widehat{g}(X) \mid X_S\,].$$

SHAP value for variable j at observation x

$$\phi_j(x) = \sum_{S \subseteq \{1,\ldots,p\} \setminus \{j\}} \frac{|S|!\,(p - |S| - 1)!}{p!}\,\big(v(S \cup \{j\}) - v(S)\big)$$

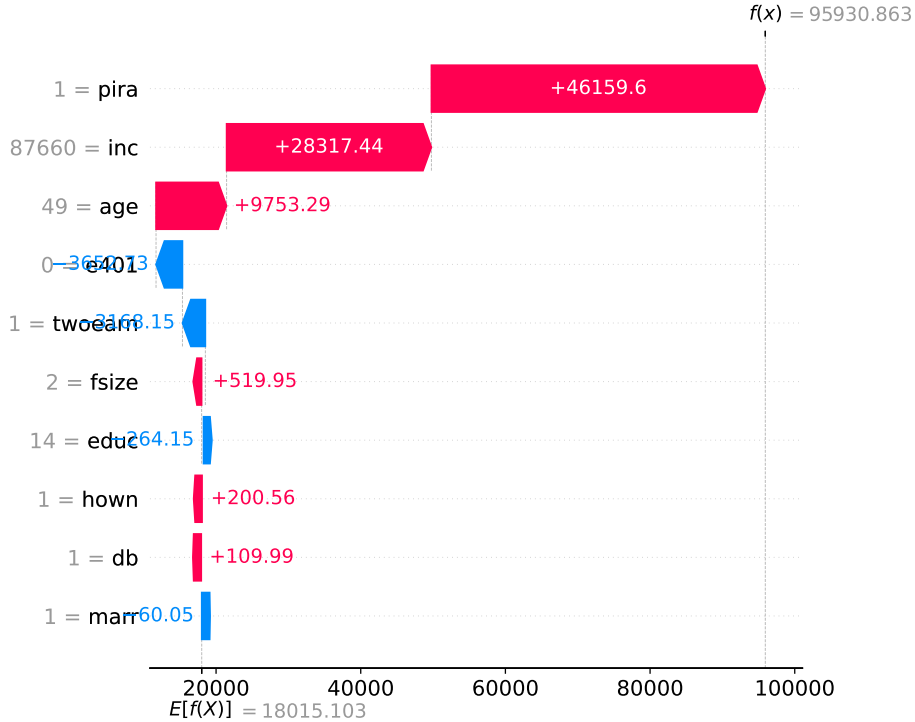
- Sum is over all subsets that do *not* include j
- $\phi_j(x)$ is the average marginal contribution of j across all coalitions
- Direct computation is $O(2^p)$ – intractable beyond toy examples
  - Practical implementations use model-specific shortcuts (e.g. *TreeSHAP* for tree ensembles, *DeepSHAP* for neural nets)

SHAP: Local Decomposition of a Prediction

SHAP values *exactly* decompose any prediction:

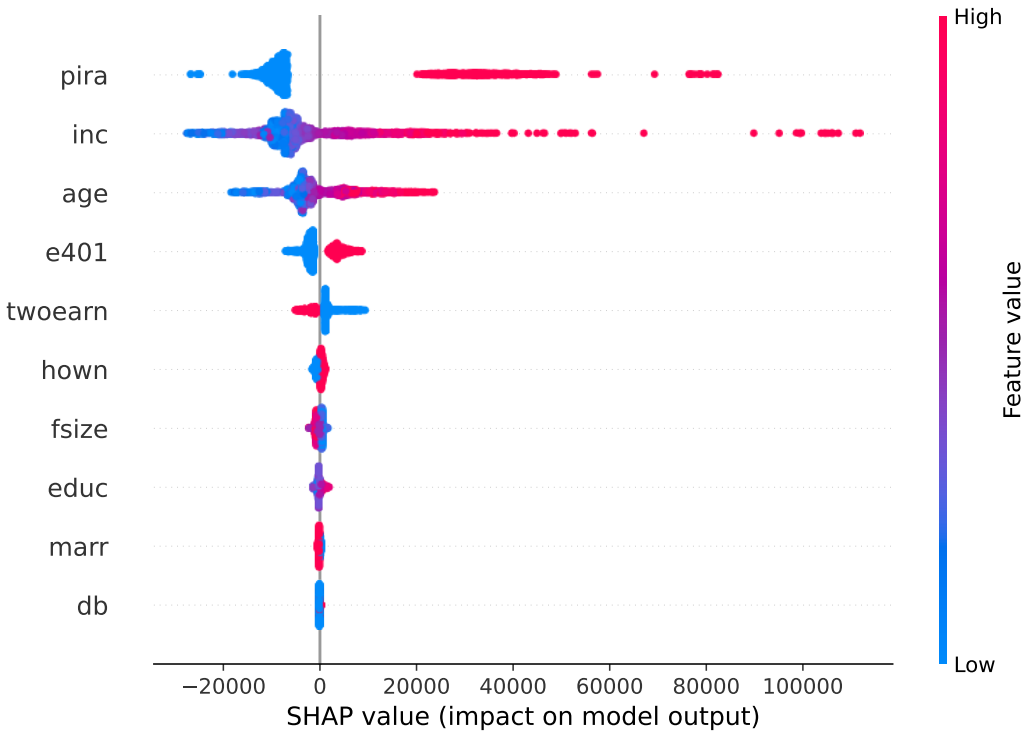
$$\widehat{g}(x_i) \;=\; \mathbb{E}_n[\widehat{g}(X)] \;+\; \sum_{j=1}^{p} \phi_j(x_i)$$

- Baseline $\mathbb{E}_n[\widehat{g}(X)]$ is the average model prediction
- Each $\phi_j(x_i)$ pushes the prediction up or down
- Different observations $\Rightarrow$ different SHAP values



SHAP attribution for one held-out 401(k) observation (Random Forest, `min_samples_leaf`=60).

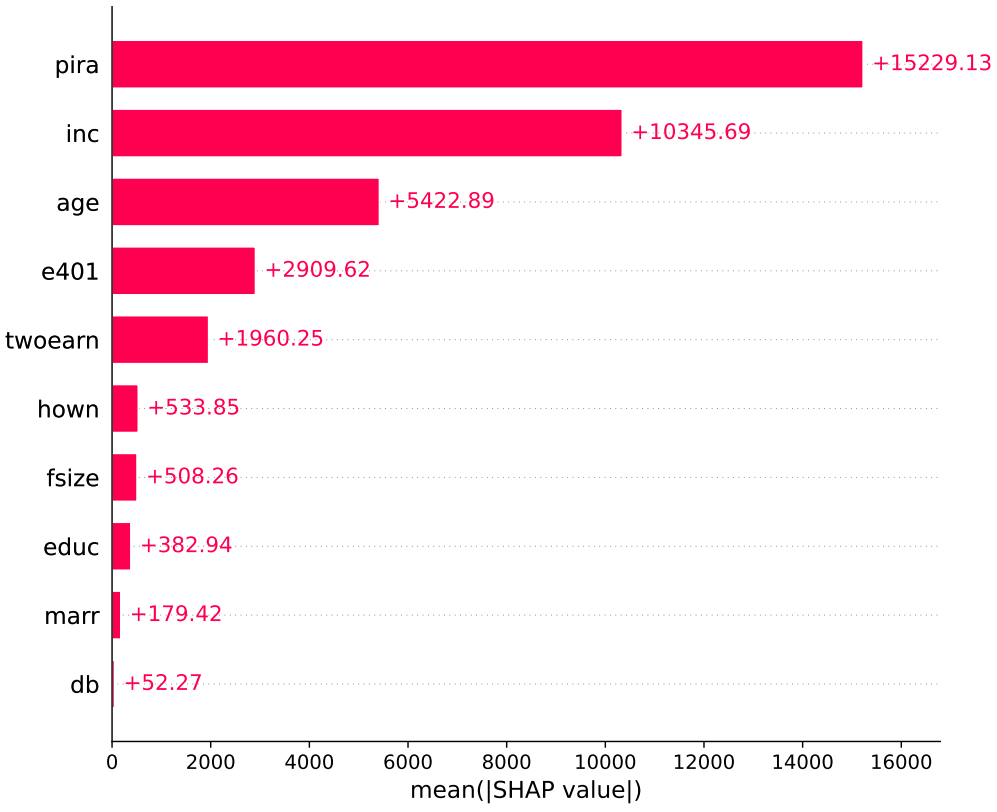
SHAP Across Observations



Each dot is one held-out obs; horizontal position is its SHAP value; color encodes variable value (low → high).

- `pira`, `inc`, `age` drive most predictions
- Spread shows *heterogeneity* – the same variable can push different observations in different directions

SHAP Variable Importance



Mean $|\phi_j|$ across the test set gives a global importance ranking:

- `pira` and `inc` dominate
- `age` is third; `e401` fourth
- Demographic controls (`marr`, `db`) close to zero

Partial Dependence

Other common interpretation device is to plot estimated function (or derivatives)

Obviously hard when dimension of X is bigger than two

Instead, report "partial dependence" of function on different X variables

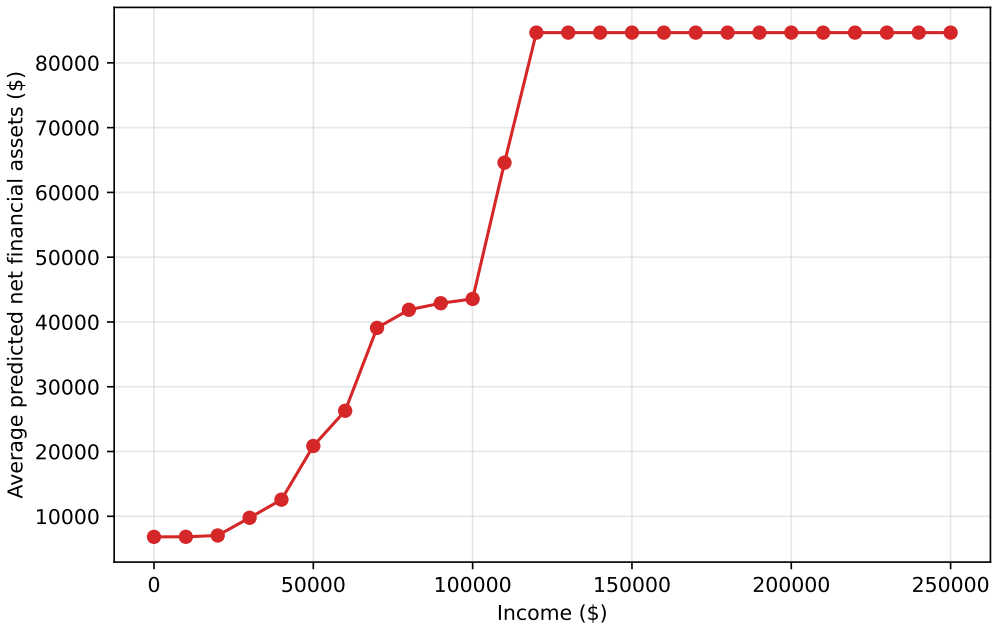
- Let X_j be a variable of interest
- Let X_{-j} be the complement of X
- Black-box prediction function: $\widehat{g}(X) = \widehat{g}(X_j, X_{-j})$
- Partial dependence is just

$$h_j(z) = \mathbb{E}_{nX_{-j}}[\widehat{g}(z, x_{-j})] = \frac{1}{m}\sum_{i=1}^{m}\widehat{f}(z, x_{i,-j})$$

  - sometimes referred to as Average Structural Function
  - could also look at derivatives (Average Marginal Effects)

Partial Dependence in Asset Example - Random Forest

Partial dependence in income



Partial dependence in `pira`

	Avg. Predicted Net Assets
`pira` = 0	8,053
`pira` = 1	35,051

Partial dependence from random forest with `min_samples_leaf`=60

Overview of Theoretical Results

Heuristic Theoretical Result

Form of Theoretical Results

Let g be a target/population prediction rule. Let $\widehat{g}$ be an estimator of g. Assume the structure of g satisfies suitable restrictions that restrict its complexity compatible with the learner $\widehat{g}$ and regularity conditions hold. Let ξ represent the complexity of g. Then

$$\|\widehat{g} - g\| \leq Cf(n, \xi) \quad \text{where} \quad f(n, \xi) \to 0$$

for an appropriate choice of $\|\cdot\|$.

General comments:

- specifics vary depending on the estimator $\widehat{g}$
- IMPORTANT: Results do not say you get the "right" g
  - estimated prediction rule is highly correlated to g
  - many ways to produce prediction rules that look like g when input variables are correlated and/or moderate strength predictors are allowed
  - e.g. no guarantees that you get the "right" variables

Flavor of Results using Lasso

- Typical Lasso convergence result looks like

$$\sqrt{\mathrm{E}_X(\beta' X - \widehat{\beta}' X)^2} \leq C\sqrt{\mathrm{E}\epsilon^2}\sqrt{\frac{s_n \log(\max\{p, n\})}{n}}$$

  - E_X denotes expectation with respect to X
  - s_n is the "effective dimension" - intuitively, number of variables with large enough coefficients to matter in sample of size n

- See, e.g., Bickel et al. (2009), Belloni and Chernozhukov (2013), Belloni et al. (2012); Belloni et al. (2016); Chernozhukov et al. (2021); Chetverikov et al. (2021); Chetverikov and Sørensen (2022) for complete statements of conditions in different settings

- $\frac{1}{n}\sum_i (x_i'\beta - x_i'\widehat{\beta})^2 = O_p(s_n \log(p)/n)$ is the best possible forecast rate
  - If you knew the right s_n regressors *ex ante*, forecast rate would be $O_p(s_n/n)$.
  - Lose $\log p$ because need to learn which regressors matter.
  - Only says your forecast rule is close to the best linear predictor $\beta' X$

References for Other Methods

Trees and related methods: See, e.g., Wager and Walther (2015), Syrgkanis and Zampetakis (2020), Ročková and van der Pas (2020)

- concentration bounds for (restricted) trees and forests
- restrictions don't actually cover the way many tree-methods are used in practice; impose only a few variables matter and target is smooth
  - Chi et al. (2022): random forests with high-dimensional data and tuning similar to common defaults can converge very slowly
  - Cattaneo et al. (2025): deep regression trees may fail to be pointwise consistent

DNN: See, e.g., Schmidt-Hieber (2020), Farrell et al. (2021), Polson and Ročková (2018)

- concentration bounds for DNNs with modest layers/neurons at global optimum
- similar restrictions on complexity of g

Theory for Modern Deep Networks

Classical results assume the learner reaches a global optimum and only model complexity is restricted. Modern DNNs

- typically highly overparameterized
- trained by SGD without global-optimum guarantees

Recent literature aims to relax both restrictions

Highly overparameterized regimes – theory accounting for the optimization path:

- Montanari and Zhong (2022), Dou and Liang (2021) – “lazy training” / NTK / data-adaptive kernel views of overparameterized networks
- Nakkiran et al. (2021) – empirical *double-descent*

Transformers and scaling laws:

- Yun et al. (2020) – transformer self-attention is a universal approximator for sequence-to-sequence functions
- Kaplan et al. (2020), Hoffmann et al. (2022) – empirical scaling laws relating loss to model size, training data, and compute

Conclusion

Commentary

Some concluding thoughts on takeaways:

- rarely know the right specification/model/learner
  - try several
  - use sensible aggregates/ensembles (essentially the “super-learner” of van der Laan and Rose (2011))
    - accurately report what you’ve done

Lots of things we didn’t cover:

- “Classical” topics: SVMs, kernel methods, Bayesian methods, classification-focused ML, matrix completion / factor models, clustering (k-means, hierarchical), AutoML
- Recent innovations: Foundation models (LLMs, vision, audio), multimodal models, Retrieval-Augmented Generation (RAG), diffusion models (generative models for images, audio, and video) being adapted to synthesize tabular data

Software used

Python (shared by R and Stata via reticulate / Stata's Python integration):

- scikit-learn (Pedregosa et al., 2011) – classical ML
- PyTorch – DNNs, transfer learning, attention
- Hugging Face `transformers` / `datasets` – pretrained models + corpora
- shap (Lundberg and Lee, 2017) – model interpretation

R: `reticulate` bridges R to the Python env for the cross-language examples; native packages (`randomForest`, `glmnet`, `treeshap`, `shapviz`)

Stata:

- `lassopack` (Ahrens et al., 2020) – regularized regression
- `pystacked` (Ahrens et al., 2023) – stacking and many ML methods via Stata's Python interface

References

Achim Ahrens, Christian B. Hansen, and Mark E. Schaffer. lassopack: Model selection and prediction with regularized regression in stata. *The Stata Journal*, 20(1):176–235, 2020. doi: 10.1177/1536867X20909697. URL https://doi.org/10.1177/1536867X20909697.

Achim Ahrens, Christian B. Hansen, and Mark E. Schaffer. pystacked: Stacking generalization and machine learning in stata. *The Stata Journal*, 23(4):909–931, 2023.

Susan Athey and Guido Imbens. Machine learning methods that economists should know about. *Annual Review of Economics*, 11:685–725, 2019.

Alexandre Belloni and Victor Chernozhukov. Least squares after model selection in high-dimensional sparse models. *Bernoulli*, 19(2):521–547, 2013. ArXiv, 2009.

Alexandre Belloni, Daniel L. Chen, Victor Chernozhukov, and Christian B. Hansen. Sparse models and methods for optimal instruments with an application to eminent domain. *Econometrica*, 80 (6):2369–2429, 2012. Arxiv, 2010.

Alexandre Belloni, Victor Chernozhukov, Christian Hansen, and Damian Kozbur. Inference in high-dimensional panel models with an application to gun control. *Journal of Business and Economic Statistics*, 34(4):590–605, 2016.

Peter J. Bickel, Ya'acov Ritov, and Alexandre B. Tsybakov. Simultaneous analysis of lasso and dantzig selector. *Annals of Statistics*, 37(4):1705–1732, 2009.

Jennie E. Brand, Xiang Zhou, and Yu Xie. Recent developments in causal inference and machine learning. *Annual Review of Sociology*, 49:81–110, 2023. doi: 10.1146/annurev-soc-030420-015345.

Leo Breiman. Bagging predictors. *Machine learning*, 26:123–140, 1996.

Leo Breiman. Random forests. *Machine learning*, 45(1):5–32, 2001.

Matias D. Cattaneo, Jason M. Klusowski, and Ruiqi Rae Yu. The honest truth about causal trees: Accuracy limits for heterogeneous treatment effect estimation. *arXiv:2509.11381*, 2025.

Victor Chernozhukov, Wolfgang Karl Härdle, Chen Huang, and Weining Wang. Lasso-driven inference in time and space. *The Annals of Statistics*, 49(3):1702–1735, 2021.

Denis Chetverikov and Jesper Riis-Vestergaard Sørensen. Analytic and bootstrap-after-cross-validation methods for selecting penalty parameters of high-dimensional m-estimators. *arXiv preprint arXiv:2104.04716*, 2022.

Denis Chetverikov, Zhipeng Liao, and Victor Chernozhukov. On cross-validated lasso in high dimensions. *Annals of Statistics*, 49(3):1300–1317, 2021.

Chien-Ming Chi, Patrick Vossler, Yingying Fan, and Jinchi Lv. Asymptotic properties of high-dimensional random forests. *Annals of Statistics*, 50(6):3415–3438, 2022.

Melissa Dell. Deep learning for economists. *Journal of Economic Literature*, 63(1):5–58, 2025. doi: 10.1257/jel.20241733.

Xialiang Dou and Tengyuan Liang. Training neural networks as learning data-adaptive kernels: Provable representation and approximation benefits. *Journal of the American Statistical Association*, 116(535):1507–1520, 2021.

Max H. Farrell, Tengyuan Liang, and Sanjog Misra. Deep neural networks for estimation and inference. *Econometrica*, 89(1):181–213, 2021. URL `https://onlinelibrary.wiley.com/doi/abs/10.3982/ECTA16901`.

Jerome Friedman, Trevor Hastie, and Robert Tibshirani. Additive logistic regression: a statistical view of boosting. *Annals of Statistics*, 28:337–407, 2000.

Jerome Friedman, Trevor Hastie, and Robert Tibshirani. *The elements of statistical learning*, volume 1. Springer series in statistics New York, 2001.

Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. The MIT Press, 2016.

Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016.

Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, et al. Training compute-optimal large language models. In *Advances in Neural Information Processing Systems*, volume 35, 2022.

Daniel Hsu, Sham M. Kakade, and Tong Zhang. Random design analysis of ridge regression. In *Conference on learning theory*, volume 23, pages 9.1–9.24. JMLR Workshop and Conference Proceedings, 2012.

Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. *arXiv:2001.08361*, 2020.

Patrick Kidger and Terry Lyons. Universal Approximation with Deep Narrow Networks. In Jacob Abernethy and Shivani Agarwal, editors, *Proceedings of Thirty Third Conference on Learning Theory*, volume 125 of *Proceedings of Machine Learning Research*, pages 2306–2327. PMLR, 09–12 Jul 2020. URL `https://proceedings.mlr.press/v125/kidger20a.html`.

Anton Korinek. Generative AI for economic research: Use cases and implications for economists. *Journal of Economic Literature*, 61(4):1281–1317, 2023. doi: 10.1257/jel.20231736.

Guillaume Lecué and Charles Mitchell. Oracle inequalities for cross-validation type procedures. *Electronic Journal of Statistics*, 6:1803–1837, 2012.

Zhou Lu, Hongming Pu, Feicheng Wang, Zhiqiang Hu, and Liwei Wang. The expressive power of neural networks: A view from the width. In *Proceedings of the 31st International Conference on Neural Information Processing Systems*, NIPS'17, pages 6232–6240, Red Hook, NY, USA, 2017. Curran Associates Inc. ISBN 9781510860964.

Scott M. Lundberg and Su-In Lee. A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

Hrushikesh Mhaskar, Qianli Liao, and Tomaso Poggio. When and why are deep networks better than shallow ones? In *Proceedings of the Thirty-First AAAI Conference on Artificial Intelligence*, AAAI'17, pages 2343–2349. AAAI Press, 2017.

Andrea Montanari and Yiqiao Zhong. The interpolation phase transition in neural networks: Memorization and generalization under lazy training. *Annals of Statistics*, 50(5):2816–2847, 2022.

Sendhil Mullainathan and Jann Spiess. Machine learning: An applied econometric approach. *Journal of Economic Perspectives*, 31(2):87–106, 2017.

Preetum Nakkiran, Gal Kaplun, Yamini Bansal, Tristan Yang, Boaz Barak, and Ilya Sutskever. Deep double descent: Where bigger models and more data hurt. *Journal of Statistical Mechanics: Theory and Experiment*, 2021(12):124003, 2021.

F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

Nicholas G Polson and Veronika Ročková. Posterior concentration for sparse deep learning. *Advances in Neural Information Processing Systems*, 31, 2018.

James M Poterba, Steven F Venti, and David A Wise. Do 401 (k) contributions crowd out other personal saving? *Journal of Public Economics*, 58(1):1–32, 1995.

Veronika Ročková and Stéphanie van der Pas. Posterior concentration for bayesian regression trees and forests. *The Annals of Statistics*, 48(4):2108–2131, 2020.

Robert E. Schapire. The strength of weak learnability. *Machine Learning*, 5:197–227, 1990.

Johannes Schmidt-Hieber. Nonparametric regression using deep neural networks with relu activation function. *Annals of Statistics*, 48(4):1875–1897, 2020.

L. S. Shapley. A value for *n*-person games. In H. W. Kuhn and A. W. Tucker, editors, *Contributions to the Theory of Games, Volume II*, number 28 in Annals of Mathematics Studies, pages 307–317. Princeton University Press, 1953.

Vasilis Syrgkanis and Manolis Zampetakis. Estimation and inference with trees and forests in high dimensions. In Jacob Abernethy and Shivani Agarwal, editors, *Proceedings of Thirty Third Conference on Learning Theory*, volume 125 of *Proceedings of Machine Learning Research*, pages 3453–3454. PMLR, 09–12 Jul 2020.

Mark J. van der Laan and Sherri Rose. *Targeted learning: causal inference for observational and experimental data*. Springer Science & Business Media, 2011.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017. URL `https://proceedings.neurips.cc/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf`.

Stefan Wager and Guenther Walther. Adaptive concentration of regression trees, with application to random forests. *arXiv preprint arXiv:1503.06388*, 2015.

Marten Wegkamp. Model selection in nonparametric regression. *The Annals of Statistics*, 31(1): 252–273, 2003.

Dmitry Yarotsky. Error bounds for approximations with deep relu networks. *Neural Networks*, 94: 103–114, 2017.

Chulhee Yun, Srinadh Bhojanapalli, Ankit Singh Rawat, Sashank J. Reddi, and Sanjiv Kumar. Are Transformers universal approximators of sequence-to-sequence functions? In *International Conference on Learning Representations*, 2020.